Module 2:Transport Layer and Network Layer

Module 2:Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
  - select()
  - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using `ip`

Transmission Control Protocol (TCP)

Definition

The **Transmission Control Protocol (TCP)** is a **connection-oriented** and **reliable** transport-layer protocol.

TCP Characteristics

- Provides reliable process-to-process communication.
- Connection-oriented protocol.
- Defines three communication phases:
  - Connection Establishment
  - Data Transfer
  - Connection Termination (Tear-down)
- Most widely used transport-layer protocol in the Internet.

Key Point

TCP guarantees reliable data delivery by maintaining a logical connection between the communicating hosts.

How TCP Provides Reliability

Reliable Transmission

TCP combines the concepts of the **Go-Back-N (GBN)** and **Selective Repeat (SR)** protocols to achieve reliable communication.

Reliability Mechanisms Used in TCP

- Checksum for error detection.
- Retransmission of lost or corrupted packets.
- Cumulative acknowledgments.
- Selective acknowledgments (SACK).
- Timers for detecting packet loss.

Chapter Roadmap

This section first discusses the **services provided by TCP**, followed by the detailed features and operation of the protocol.

TCP Services

Services Offered by TCP

Before studying the internal operation of TCP, it is important to understand the services it provides to application-layer processes.

TCP provides the following services:

- Process-to-Process Communication
- Stream Delivery Service
- Sending and Receiving Buffers
- Segmentation
- Full-Duplex Communication
- Multiplexing and Demultiplexing
- Connection-Oriented Service
- Reliable Service

Process-to-Process Communication

Communication Between Processes

Like UDP, TCP provides process-to-process communication using **port numbers**.

- Each application process is identified by a port number.
- TCP delivers data to the correct destination process.
- Source and destination port numbers uniquely identify a communication session.
- Common TCP applications use well-known port numbers.

Key Point

TCP communicates between application processes, not directly between hosts.

Stream Delivery Service

TCP is Stream-Oriented

Unlike UDP, TCP treats application data as a continuous stream of bytes.

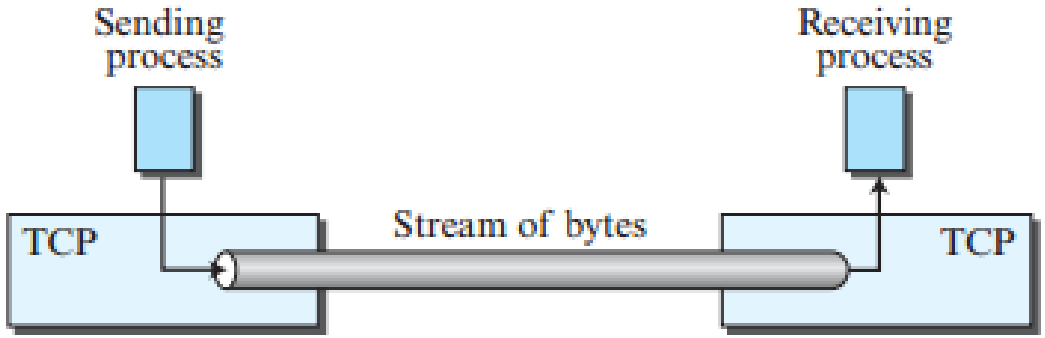
UDP

- Sends independent messages.
- Message boundaries are preserved.

TCP

- Sender writes bytes into a stream.
- Receiver reads bytes from the stream.
- No message boundaries are visible.
- Applications appear connected by an imaginary communication tube.

Figure 3.41 *Stream delivery*



Sending and Receiving Buffers

Purpose of Buffers

TCP uses buffers because sending and receiving processes may operate at different speeds.

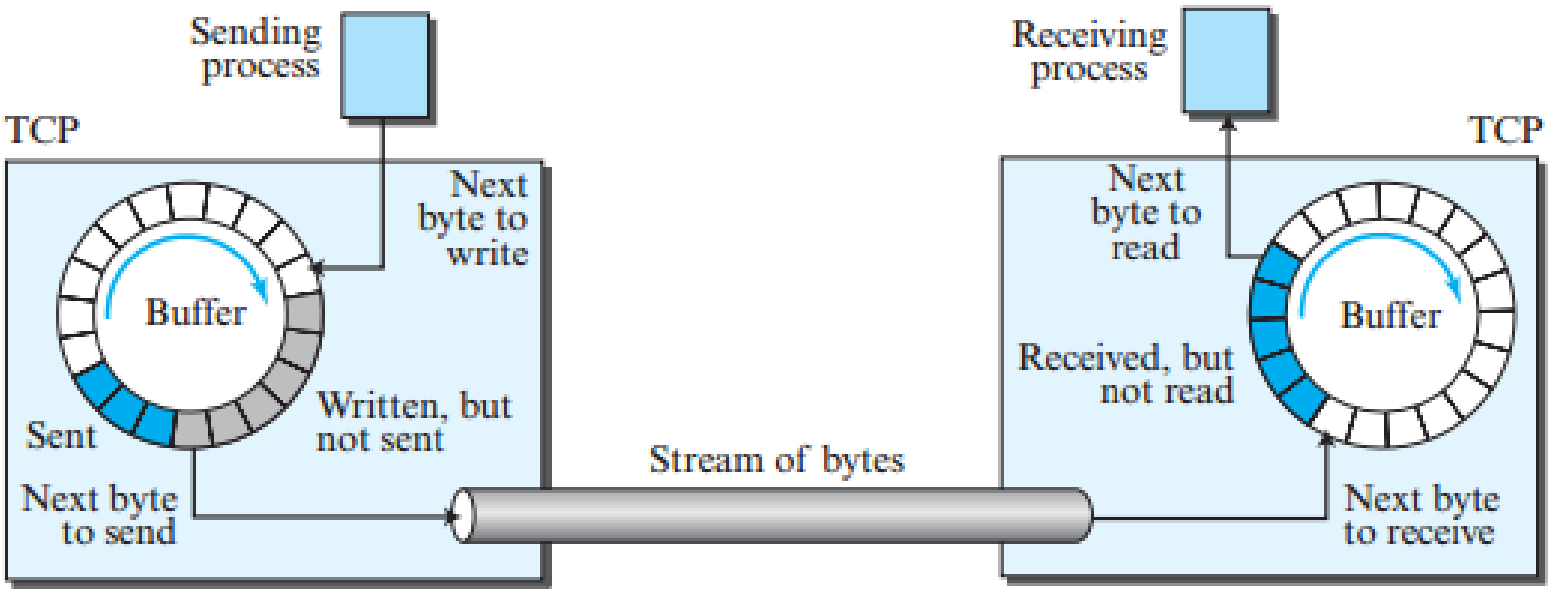
Each direction has:

- One Sending Buffer
- One Receiving Buffer

Buffers are also required for

- Flow Control
- Error Control

Figure 3.42 *Sending and receiving buffers*



Operation of TCP Buffers

Sender Buffer

- Empty chambers available for new data.
- Sent but unacknowledged bytes remain stored.
- Bytes waiting for transmission remain in the buffer.
- Acknowledged bytes are recycled.

Receiver Buffer

- Empty locations receive arriving bytes.
- Received bytes wait until the application reads them.
- Read locations are recycled for future use.

Key Point

TCP commonly implements buffers as circular arrays to efficiently reuse memory.

Segments

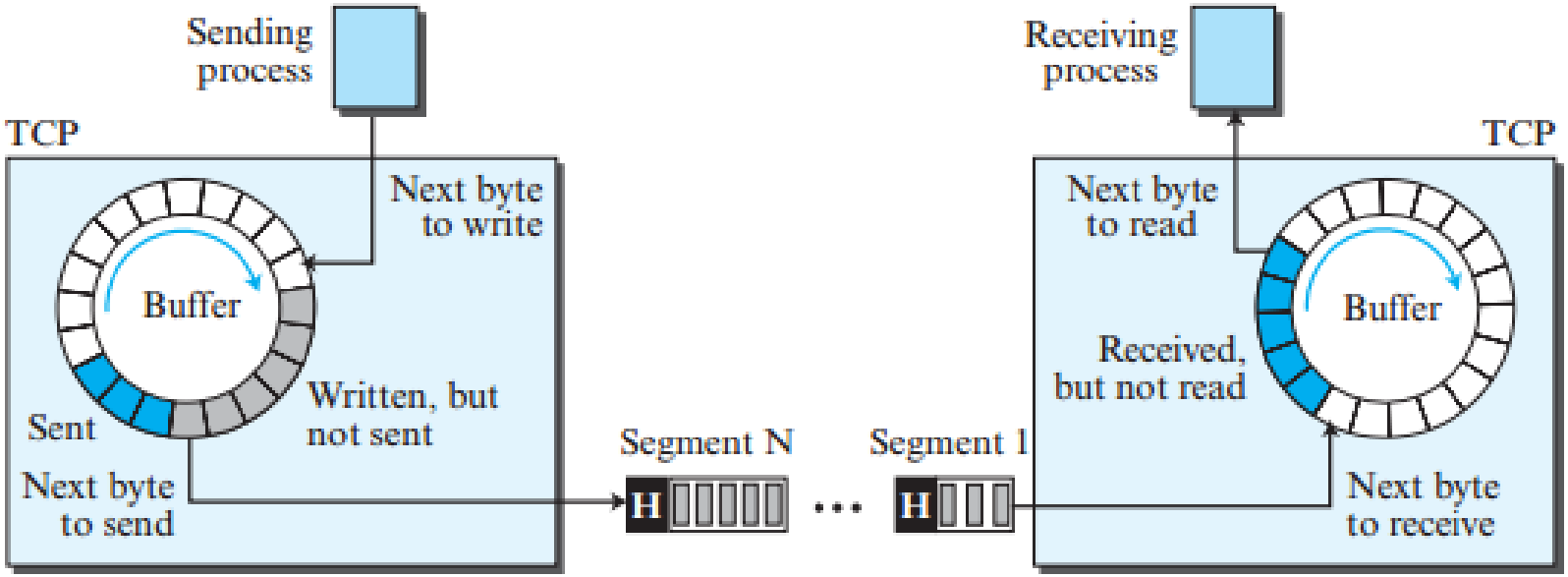
Segmentation

Although applications generate a stream of bytes, the network layer transmits packets.

TCP therefore:

- Groups bytes into packets called **segments**.
- Adds a TCP header to each segment.
- Passes each segment to IP.
- IP encapsulates the segment inside an IP datagram.

Figure 3.43 *TCP segments*



Characteristics of TCP Segments

- Segments are not necessarily equal in size.
- A segment may contain hundreds or thousands of bytes.
- Segments may arrive:
  - Out of order
  - Lost
  - Corrupted
  - Retransmitted
- These operations are completely transparent to the receiving application.

Key Point

Applications see only an ordered stream of bytes even though individual segments may travel differently through the network.

Full-Duplex Communication and Multiplexing

Full-Duplex Communication

- Data flows simultaneously in both directions.
- Each endpoint has its own sending and receiving buffers.

Multiplexing and Demultiplexing

- Multiple applications share TCP.
- Sender performs multiplexing.
- Receiver performs demultiplexing.
- A separate logical connection is established for each communicating process pair.

Connection-Oriented and Reliable Service

Connection-Oriented Communication

1. Establish a logical connection.
2. Exchange data in both directions.
3. Terminate the connection.

Reliable Service

- TCP guarantees reliable delivery.
- Uses acknowledgments to verify successful reception.
- Lost or corrupted segments are retransmitted.
- Applications receive data in the correct order.

Important

TCP establishes a **logical connection**, not a physical one. Each TCP segment is encapsulated in an IP datagram, and different segments may travel through different network paths before reaching their destination.

TCP Features —TCP Numbering System

- TCP numbers **bytes**, not segments.
- Two header fields are used:
  - Sequence Number
  - Acknowledgment Number
- These fields provide reliable communication.

Key Point

TCP reliability is based on byte numbering rather than segment numbering.

Byte Number and Sequence Number

Byte Number

- Every transmitted byte (octet) is numbered.
- Numbering is independent in each direction.
- The first byte is assigned a random value between 0 and $2^{32} - 1$.
- Used for flow control and error control.

Sequence Number

- Identifies the **first byte** carried in a segment.
- The first segment uses a random **Initial Sequence Number (ISN)**.
- Next sequence number = Previous Sequence Number + Bytes in Previous Segment.

Example 3.17: Sequence Numbers

Given

- File Size = 5000 bytes
- First Byte Number = 10001
- Five segments carrying 1000 bytes each

Segment	Sequence Number
1	10001
2	11001
3	12001
4	13001
5	14001

Control Segments

Segments used for connection establishment, termination, or abortion carry no user data but consume one sequence number as if carrying one imaginary byte.

Acknowledgment Number

Definition

The acknowledgment number specifies the **next byte expected** by the receiver.

- Communication is full duplex.
- Each side numbers its own transmitted bytes.
- Acknowledgments are cumulative.
-

$$ACK = \text{Last Correct Byte Received} + 1$$

Example

If

$$ACK = 5643$$

then the receiver has correctly received every byte up to byte number **5642**.

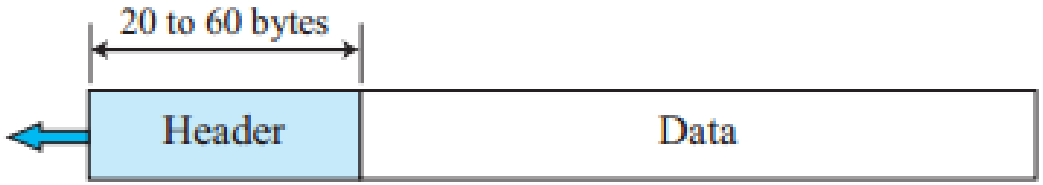
TCP Segment

Definition

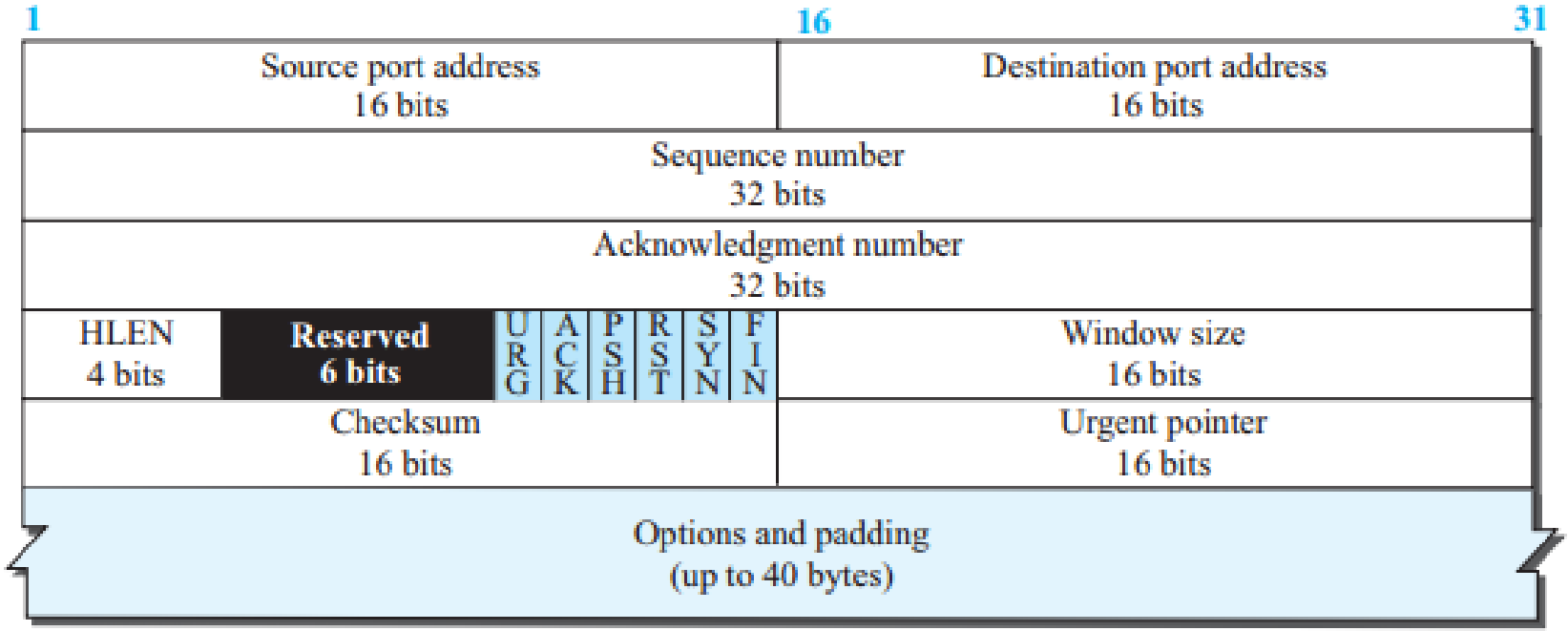
A packet in TCP is called a **segment**.

- A TCP segment consists of:
  - Header (20–60 bytes)
  - Application Data
- Header size depends on whether options are present.
- Minimum header size = 20 bytes.
- Maximum header size = 60 bytes.

Figure 3.44 *TCP segment format*



a. Segment



b. Header

TCP Header Fields

Field	**Purpose**
Source Port	Sender application port
Destination Port	Receiver application port
Sequence Number	First byte number in the segment
Acknowledgment Number	Next expected byte number
Header Length	Size of TCP header (20–60 bytes)

Key Point

TCP numbers **bytes**, not segments.

Control Flags and Window Size

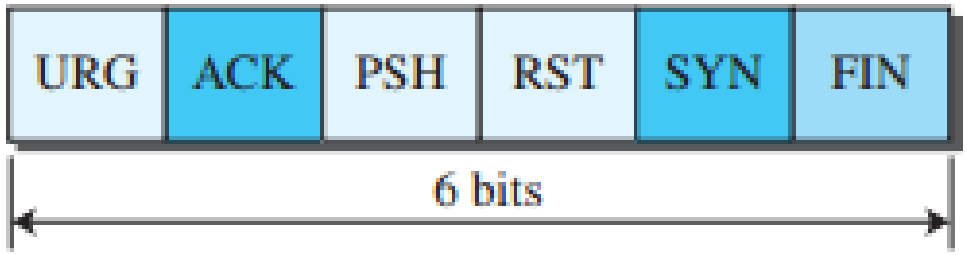
Control Flags

- TCP uses six control bits (flags).
- Used for:
  - Connection establishment
  - Connection termination
  - Connection abortion
  - Flow control
  - Data transfer control

Window Size

- 16-bit field
- Maximum window size = 65,535 bytes
- Determined by the receiver (**rwnd**)

Figure 3.45 *Control field*



URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbe
FIN: Terminate the connection

Checksum, Urgent Pointer and Options

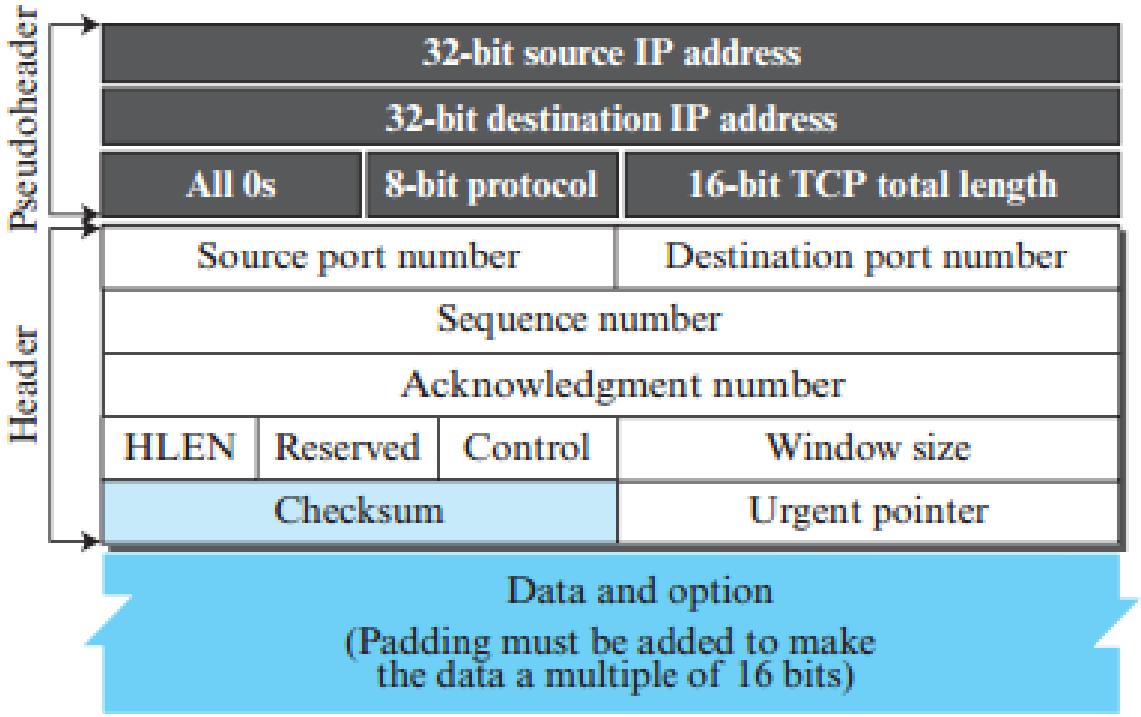
Checksum

- 16-bit mandatory field.
- Same calculation method as UDP.
- Uses a pseudoheader.
- Protocol number = 6 (TCP).

Other Fields

- Urgent Pointer: Valid only when the URG flag is set.
- Options: Up to 40 bytes of optional information.

Figure 3.46 *Pseudoheader added to the TCP datagram*



The use of the checksum in TCP is mandatory.

TCP Encapsulation

Encapsulation Process

TCP receives data from the application layer and encapsulates it into a **TCP Segment**.

Application Data → TCP Segment → IP Datagram → Data-Link Frame

- TCP segment is encapsulated inside an IP datagram.
- The IP datagram is encapsulated inside a data-link frame for transmission.

Key Point

Each layer adds its own header before forwarding the data to the next lower layer.

TCP Connection

Connection-Oriented Protocol

TCP establishes a **logical connection** between the source and destination before data transfer.

- Uses IP for segment delivery.
- TCP controls reliability.
- Lost segments are retransmitted.
- Out-of-order segments are reordered before delivery.

Key Point

TCP connection is **logical**, not physical.

Phases of a TCP Connection

TCP communication consists of three phases:

1. Connection Establishment
2. Data Transfer
3. Connection Termination

$$\text{Connection} \rightarrow \text{Data Transfer} \rightarrow \text{Termination}$$

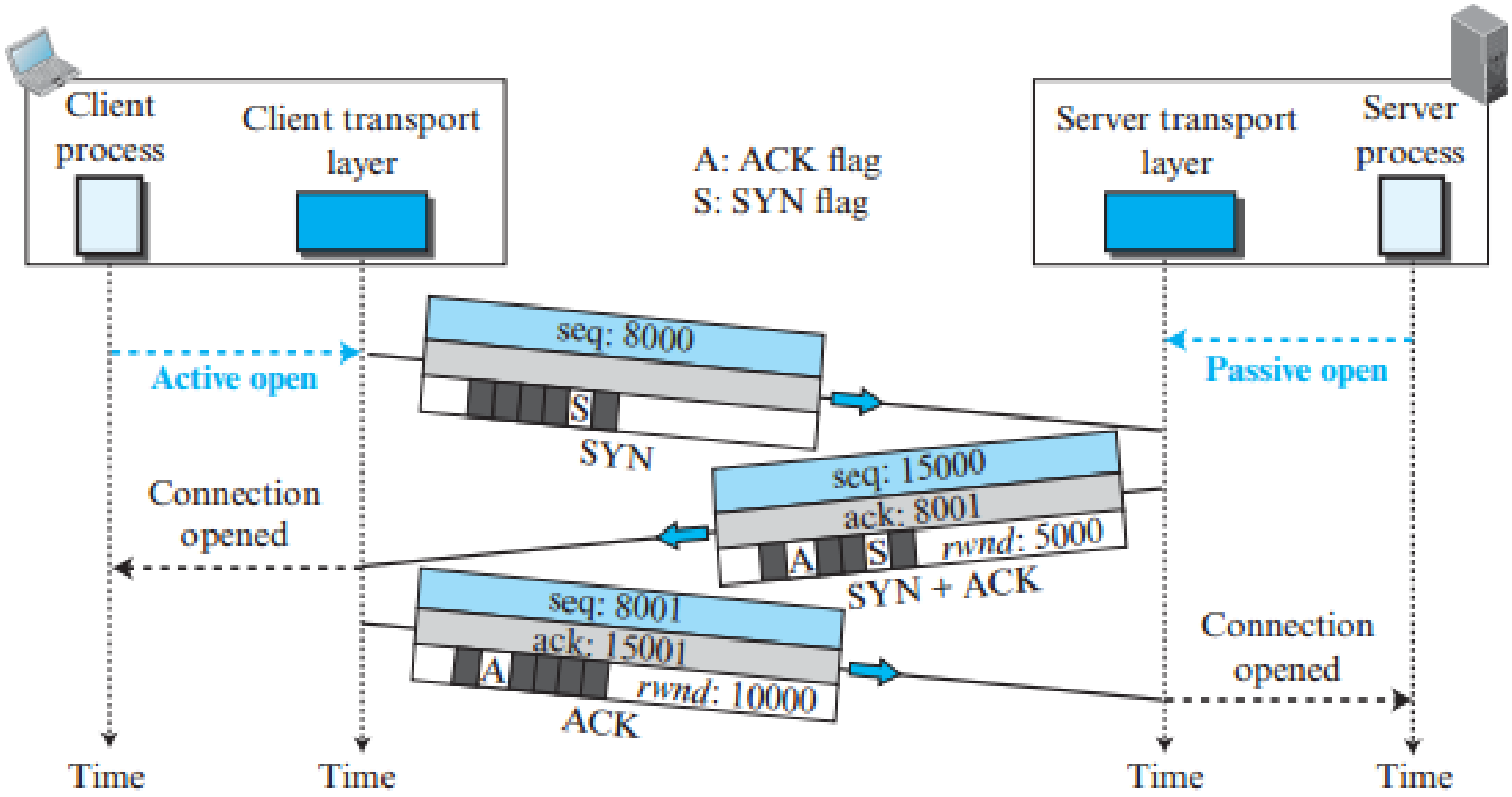
Connection Establishment

Three-Way Handshaking

TCP establishes a connection using a **three-way handshake**.

1. Client sends SYN
2. Server replies with SYN + ACK
3. Client sends ACK

Figure 3.47 *Connection establishment using three-way handshaking*



Three-Way Handshaking

Step	Purpose
SYN	Client requests connection and sends ISN.
SYN + ACK	Server acknowledges client and sends its ISN.
ACK	Client confirms the server's ISN.

- SYN consumes one sequence number.
- SYN+ACK consumes one sequence number.
- ACK consumes no sequence number unless carrying data.

SYN Flooding Attack

- Attacker sends many fake SYN requests.
- Server allocates resources for each request.
- Final ACK never arrives.
- Resources become exhausted.
- Valid users may be denied service.

Protection

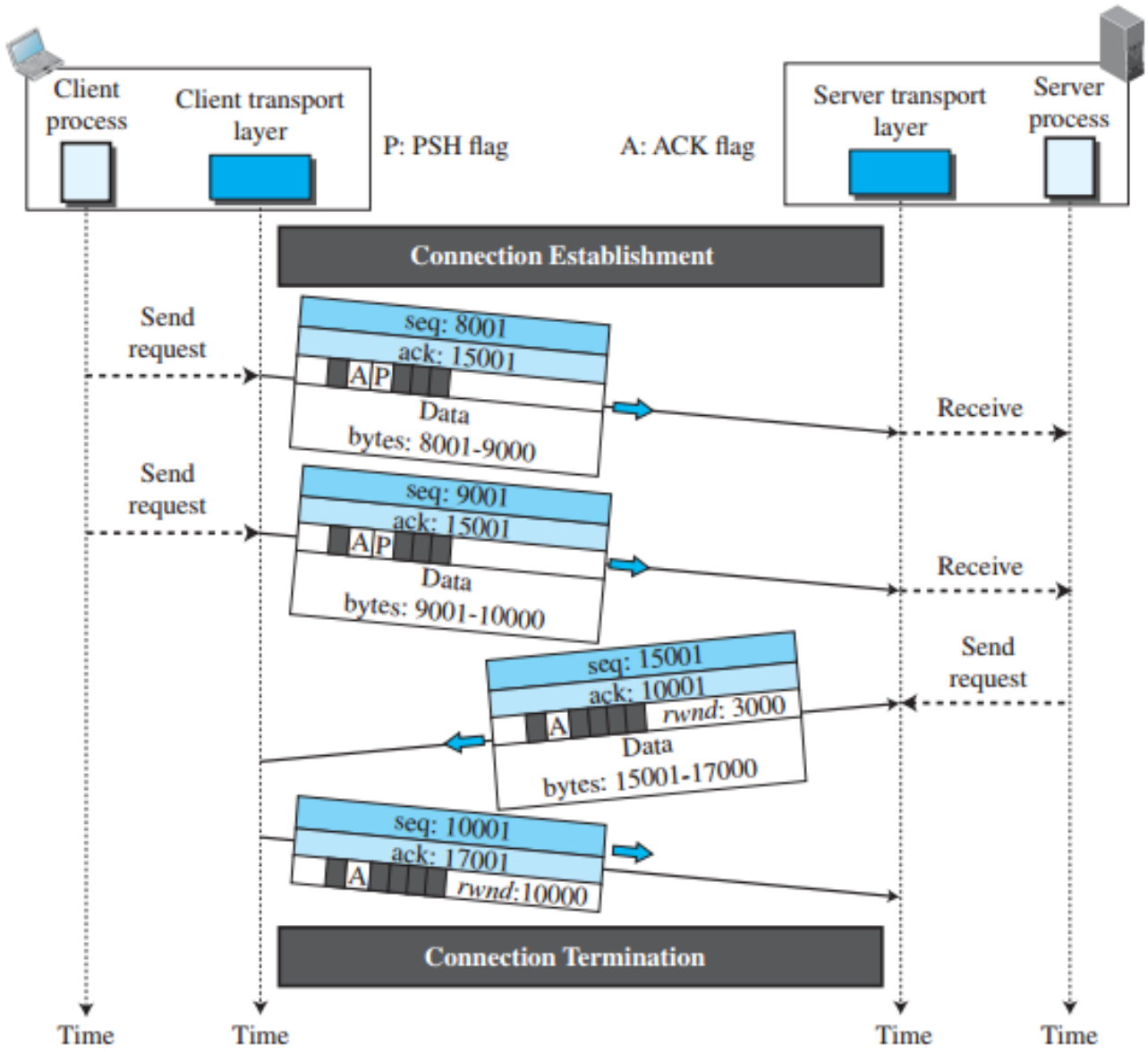
- Limit pending connections.
- Filter suspicious requests.
- Use SYN Cookies.

Data Transfer

After connection establishment,

- Client and server communicate simultaneously.
- TCP supports full-duplex communication.
- Data and acknowledgments are usually piggybacked.
- PSH flag delivers data immediately to the application.

Figure 3.48 *Data transfer*



Pushing Data and Urgent Data

Push (PSH)

- Sends buffered data immediately.
- Receiver delivers data immediately.

Urgent Data (URG)

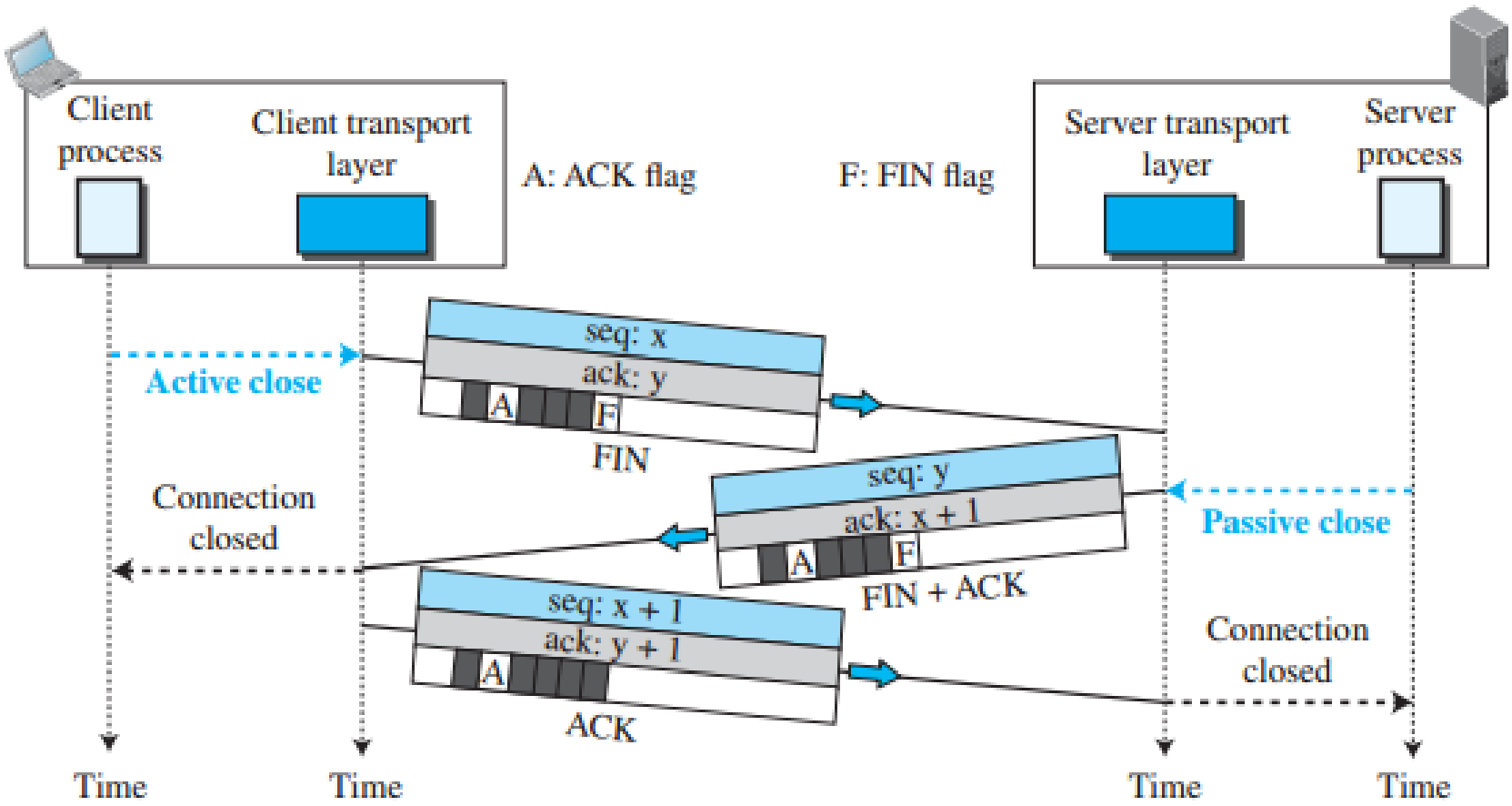
- Marks urgent bytes.
- Uses the Urgent Pointer field.
- Receiver identifies urgent data.
- Application decides how to process it.

Connection Termination

TCP normally closes a connection using **three-way handshaking**.

1. Client sends FIN
2. Server replies with FIN + ACK
3. Client sends ACK

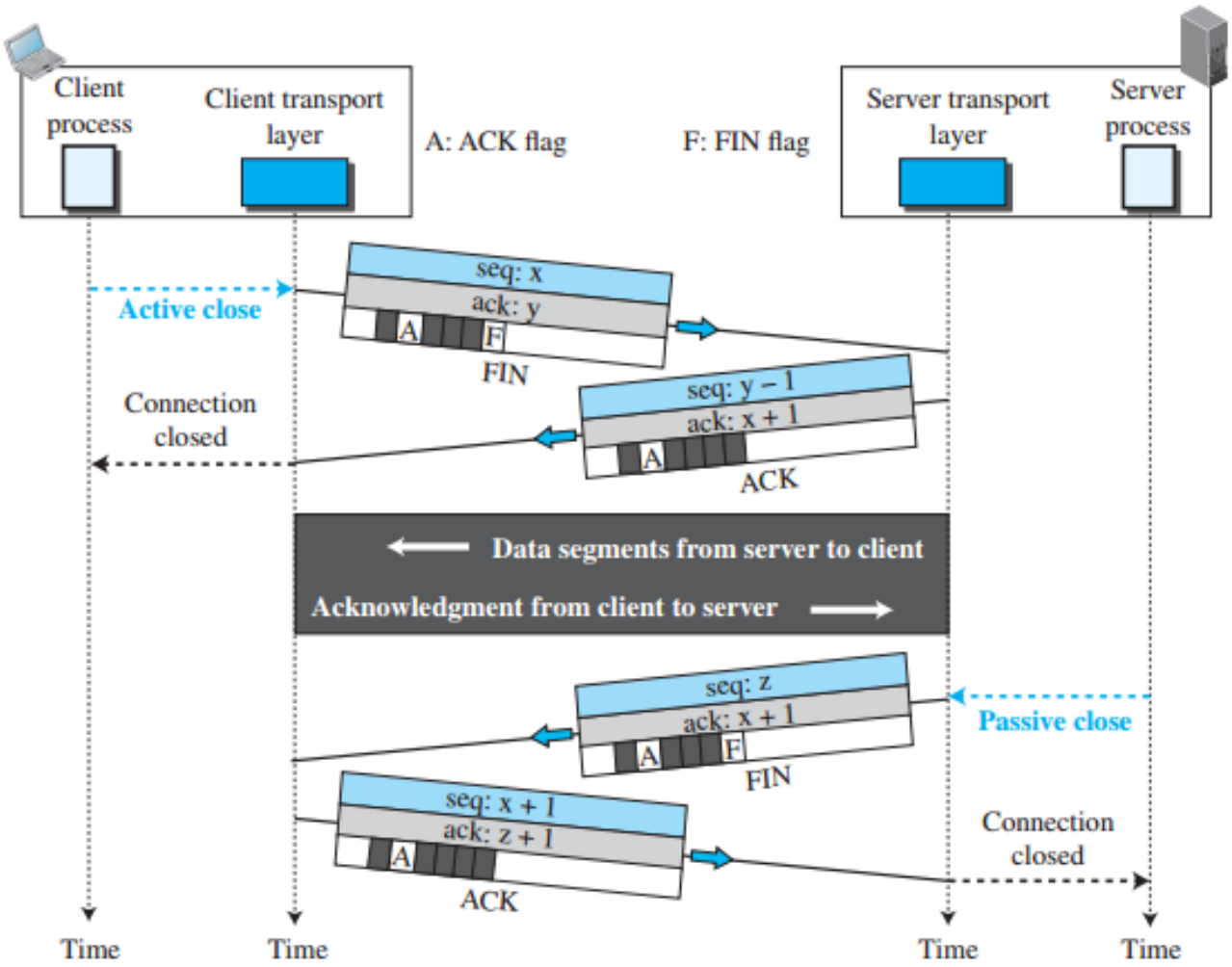
Figure 3.49 *Connection termination using three-way handshaking*



Half-Close Connection

- One direction is closed.
- The opposite direction remains active.
- Client can stop sending while still receiving.
- Common in applications such as file sorting.

Figure 3.50 *Half-close*



Connection Reset

RST (Reset)

TCP uses the **RST** flag to immediately terminate or reject a connection.

- Reject a connection request.
- Abort an existing connection.
- Terminate an idle connection.

Summary

- Three phases of TCP communication.
- Three-way handshake for connection setup.
- Full-duplex data transfer.
- PSH and URG support special transmission.
- FIN and ACK terminate the connection.
- RST immediately resets a connection.

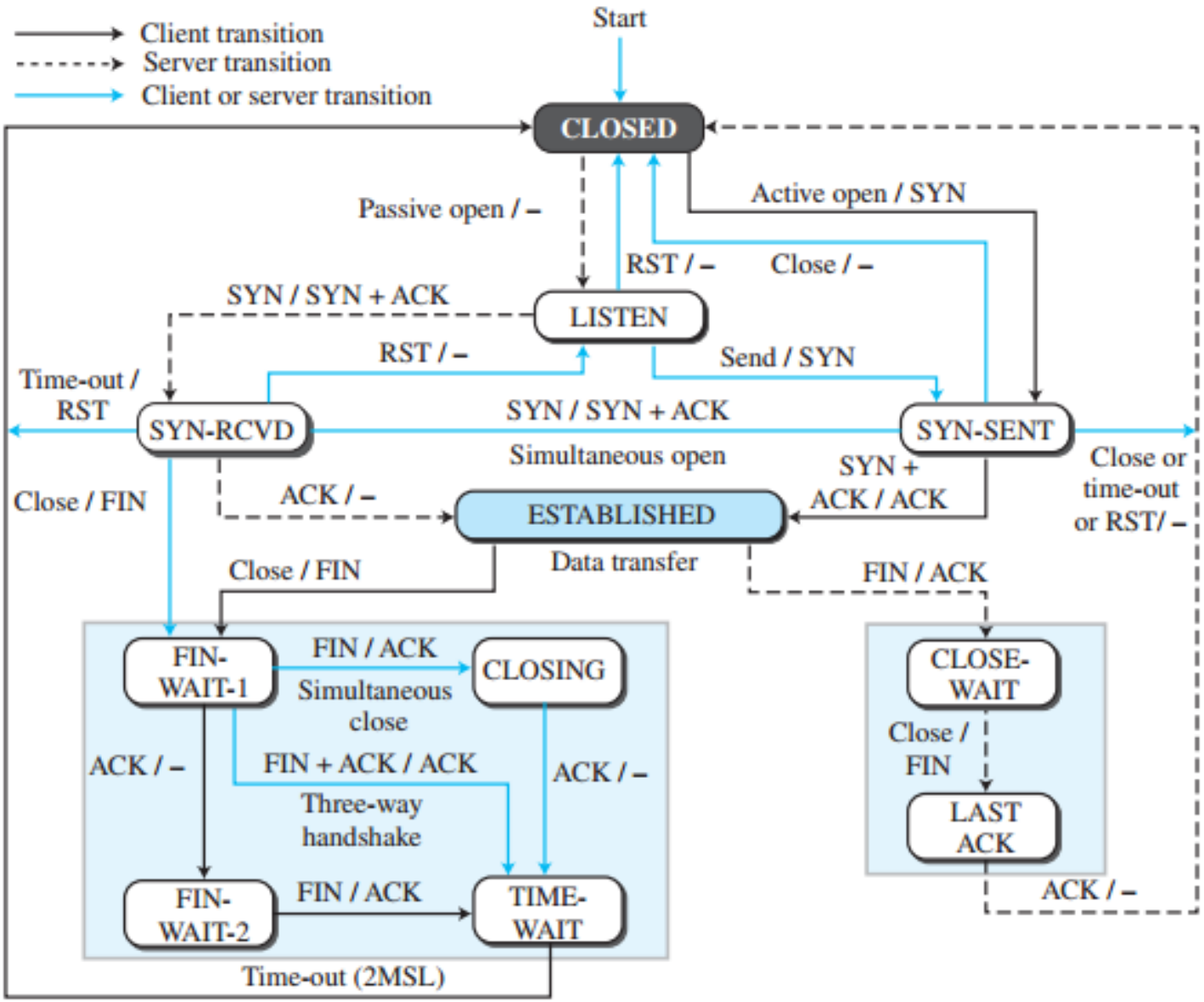
TCP State Transition Diagram

Finite State Machine (FSM)

TCP is specified as a **Finite State Machine (FSM)** to track connection establishment, data transfer, and connection termination.

- States are represented by rounded rectangles.
- Directed arrows represent state transitions.
- Each transition shows:
  - Input received
  - Output generated
- Separate state transitions exist for both client and server.

Figure 3.51 *State transition diagram*



TCP States

State	Description
CLOSED	No connection exists
LISTEN	Waiting for SYN request
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN+ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer
FIN-WAIT-1 / FIN-WAIT-2	Waiting for connection termination
CLOSE-WAIT	Waiting for application to close
LAST-ACK	Waiting for final ACK
TIME-WAIT	Waiting for 2MSL timeout
CLOSING	Simultaneous close by both ends

Normal TCP State Flow

Client

1. CLOSED
2. SYN-SENT
3. ESTABLISHED
4. FIN-WAIT
5. TIME-WAIT
6. CLOSED

Server

1. CLOSED
2. LISTEN
3. SYN-RCVD
4. ESTABLISHED
5. CLOSE-WAIT
6. LAST-ACK
7. CLOSED

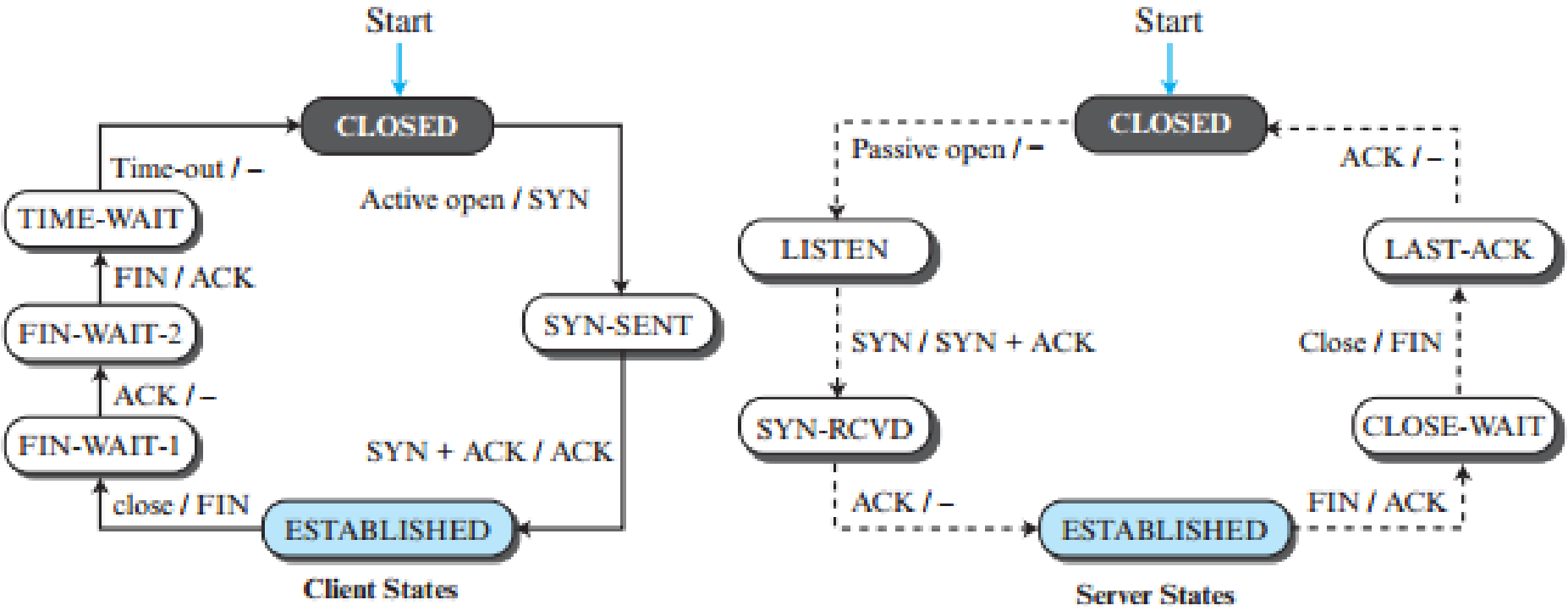
Half-Close Scenario

Half-Close

One side stops sending data while continuing to receive data.

- Client performs an active close.
- Client sends FIN and enters FIN-WAIT state.
- Server acknowledges FIN and enters CLOSE-WAIT state.
- Server continues sending remaining data.
- Server finally sends FIN.
- Client replies with ACK and enters TIME-WAIT.

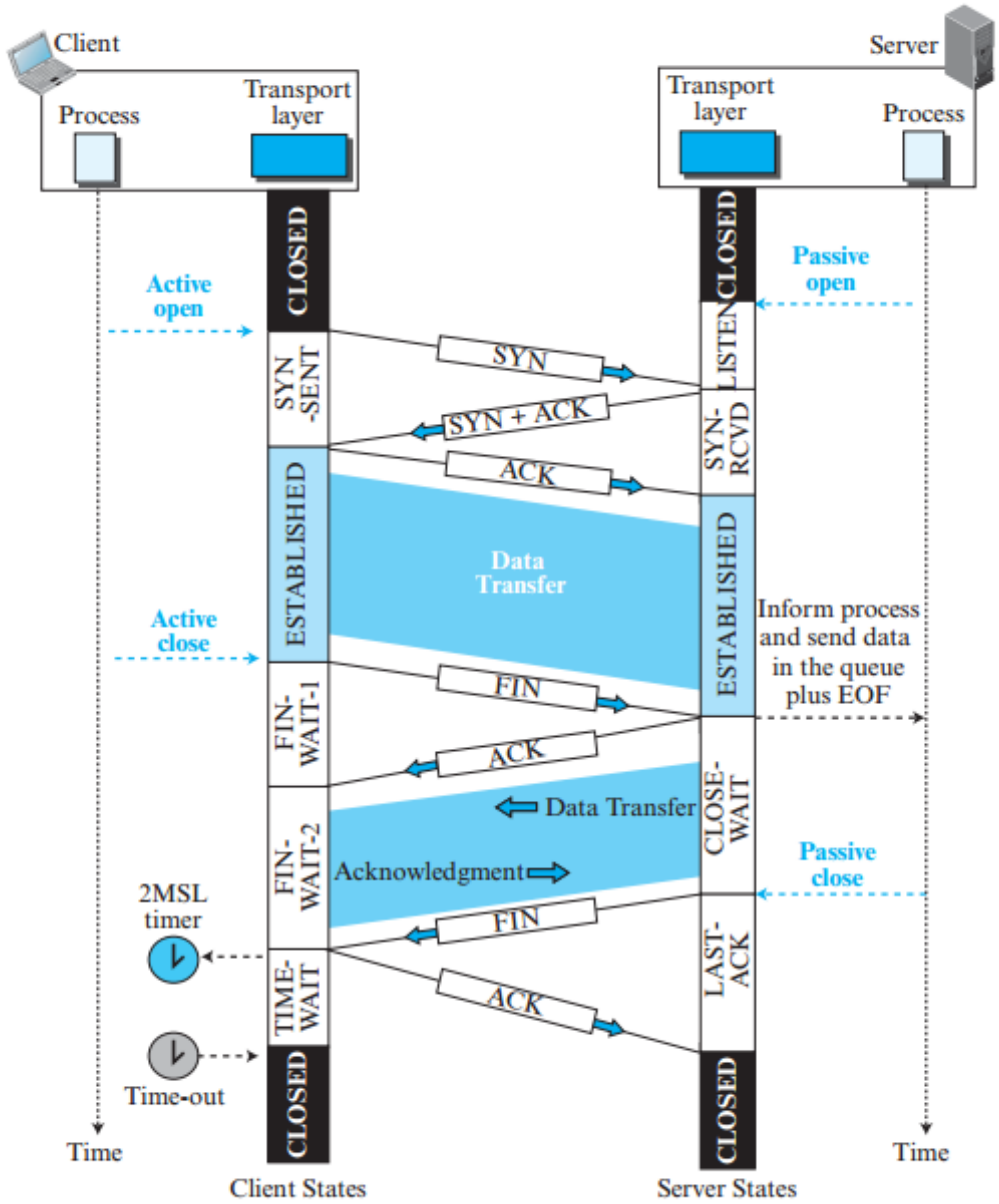
Figure 3.52 *Transition diagram with half-close connection termination*



Half-Close State Transition

- Client:
  - Active Open
  - ESTABLISHED
  - Active Close
  - FIN-WAIT-1
  - FIN-WAIT-2
  - TIME-WAIT
  - CLOSED
- Server:
  - Passive Open
  - LISTEN
  - SYN-RCVD
  - ESTABLISHED
  - CLOSE-WAIT
  - LAST-ACK
  - CLOSED

Figure 3.53 *Time-line diagram for a common scenario*



Windows in TCP

TCP Windows

TCP uses windows to manage reliable data transmission, flow control, and congestion control.

- Two windows are used in each direction:
  - Send Window
  - Receive Window
- Bidirectional communication requires **four windows** (two at each endpoint).
- For simplicity, TCP window operation is usually explained using one-way communication.

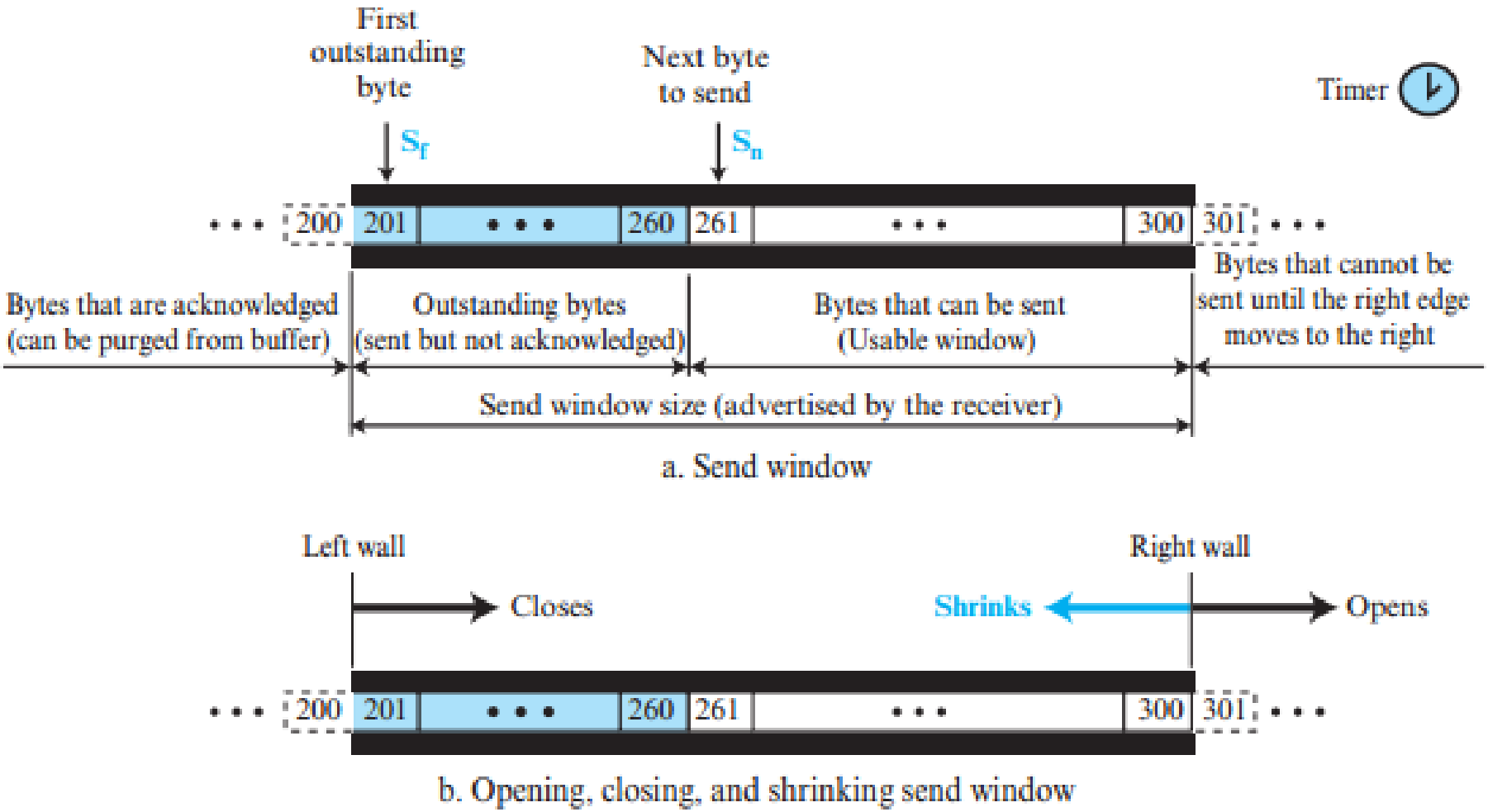
Send Window

- The send window determines how many bytes can be transmitted.
- Window size is controlled by:
  - Receiver (Flow Control)
  - Network Congestion (Congestion Control)
- The send window opens, closes, and may shrink.

Differences from Selective Repeat (SR)

1. Window size is measured in **bytes**, not packets.
2. TCP may buffer application data before transmission.
3. TCP uses only **one timer** instead of multiple timers.

Figure 3.54 *Send window in TCP*

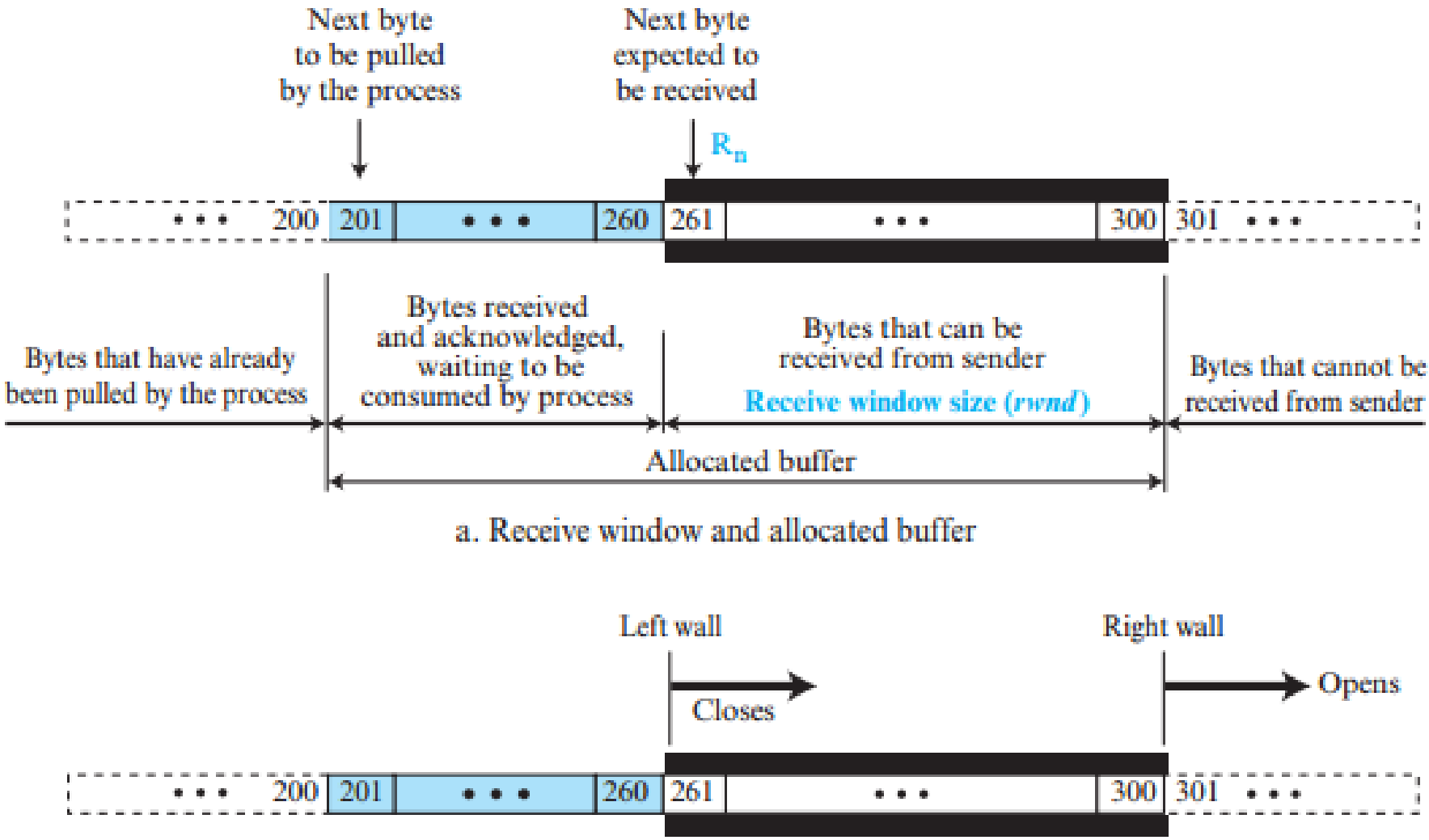


Receive Window

- The receive window determines how many bytes can be accepted.
- Received data remain in the buffer until read by the application.
- The receive window usually opens and closes but should never shrink.

rwnd = Buffer Size - Waiting Bytes

Figure 3.55 *Receive window in TCP*



TCP Receive Window vs Selective Repeat

TCP Receive Window

- Application reads data at its own speed.
- Receive window depends on available buffer space.
- Uses cumulative acknowledgments.
- Newer TCP also supports Selective ACK (SACK).

Selective Repeat

- Window measured in packets.
- Uses selective acknowledgments.
- Multiple timers.
- Receiver acknowledges individual packets.

Key Point

TCP primarily uses **cumulative acknowledgments** (next expected byte), while modern TCP can also use **Selective Acknowledgments (SACK)**.

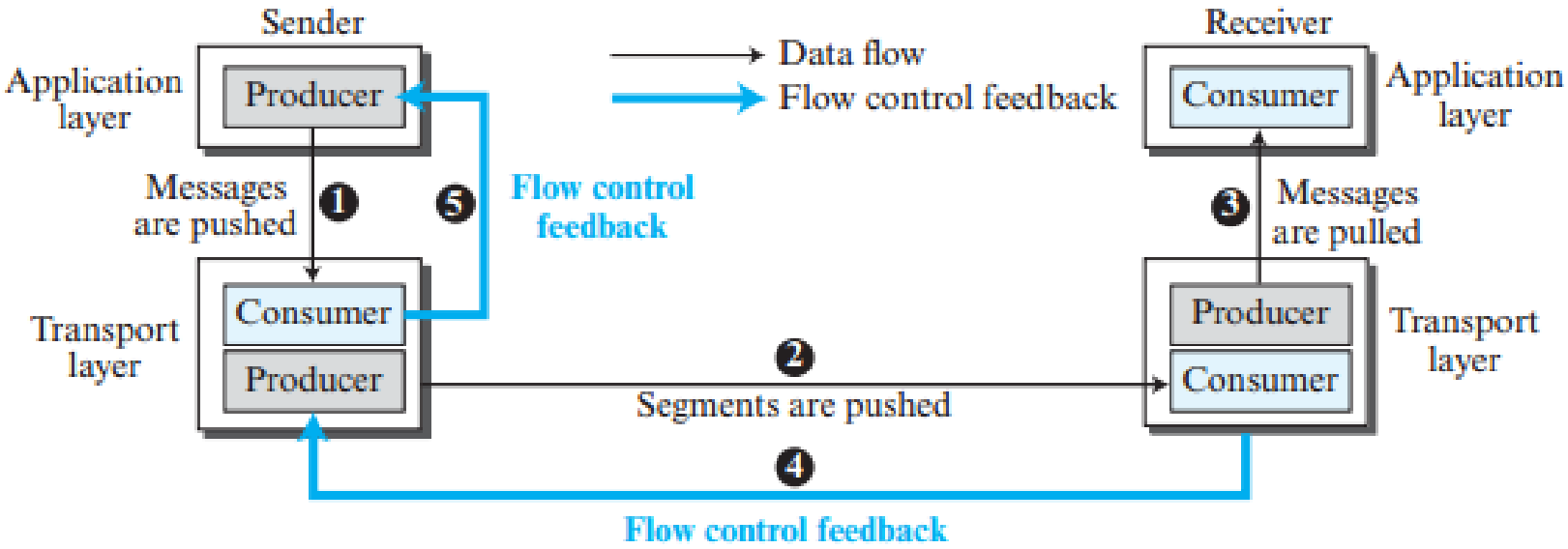
TCP Flow Control

Flow Control

Flow control balances the rate at which the sender produces data with the rate at which the receiver can consume it.

- TCP separates **Flow Control** from **Error Control**.
- Assumes an error-free logical channel.
- Feedback is sent from the receiving TCP to the sending TCP.
- The sender adjusts its transmission rate according to the receiver.

Figure 3.56 *Data flow and flow control feedbacks in TCP*



Opening and Closing Windows

Receive Window

- Closes when new bytes arrive.
- Opens when the application reads data.
- Normally should never shrink.

Send Window

- Closes when ACKs are received.
- Opens when receiver advertises a larger **rwnd**.
- May shrink only in special situations.

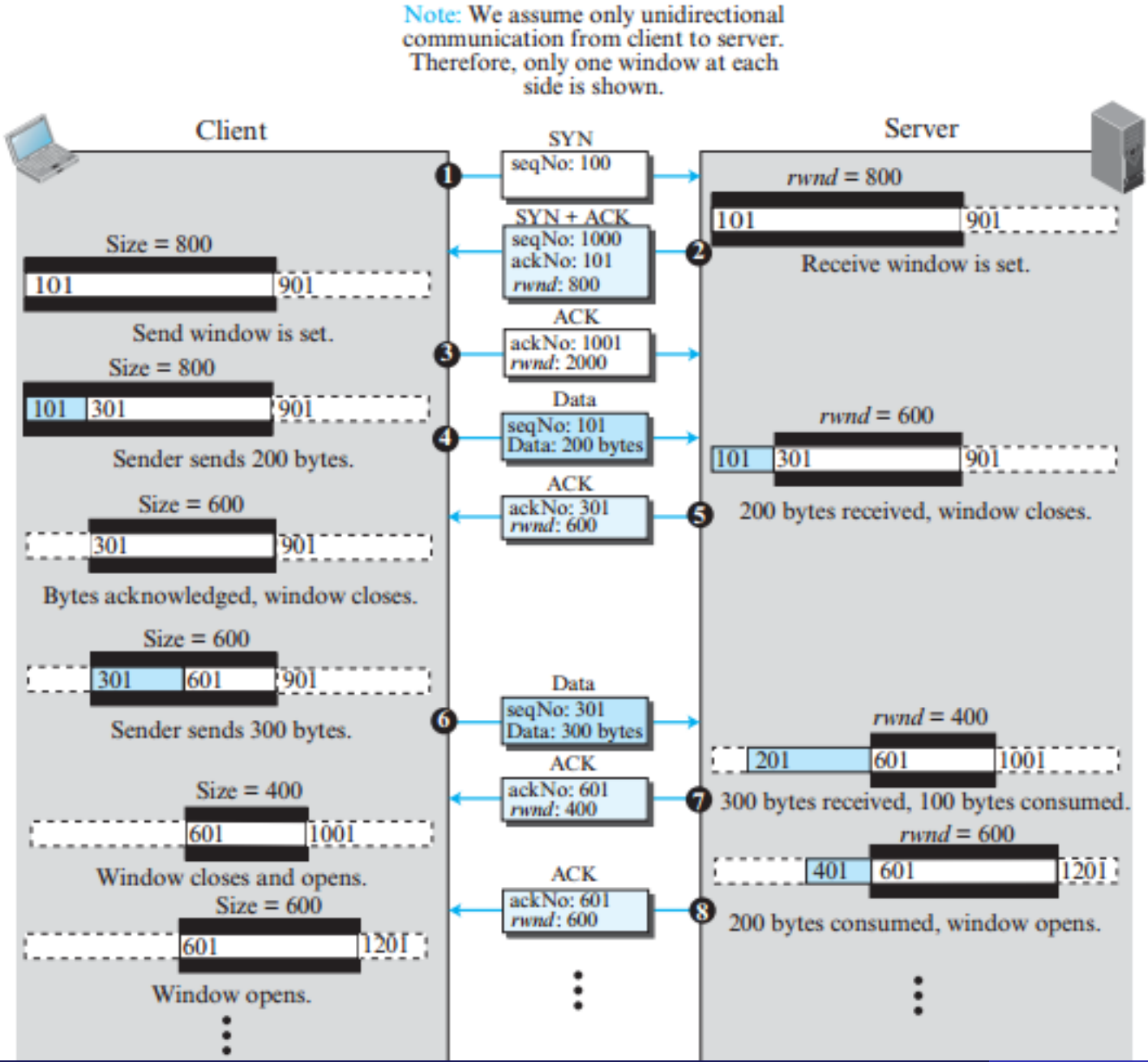
Key Point

The receiver controls the sender by advertising the **receive window (rwnd)**.

Flow Control Scenario

- Connection established with receiver window (**rwnd**).
- Sender transmits data within the advertised window.
- Receiver stores data and sends ACK + updated **rwnd**.
- Sender adjusts its send window accordingly.
- Window size increases or decreases based on available buffer space.

Figure 3.57 *An example of flow control*



Window Shrinking and Window Shutdown

Window Shrinking

- Receive window should never shrink.
- Send window shrinking is discouraged.
- Receiver should satisfy

$$\boxed{\text{new ACK} + \text{new rwnd} \geq \text{old ACK} + \text{old rwnd}}$$

to avoid shrinking.

Window Shutdown

- Receiver may advertise **rwnd = 0**.
- Sender temporarily stops transmission.
- Sender periodically sends a **1-byte probe** to prevent deadlock.

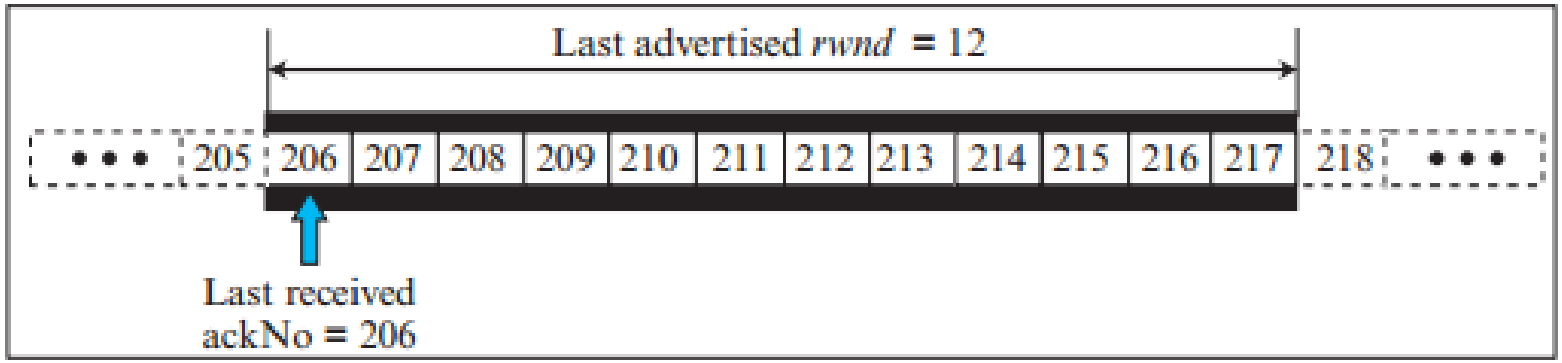
Silly Window Syndrome

Problem

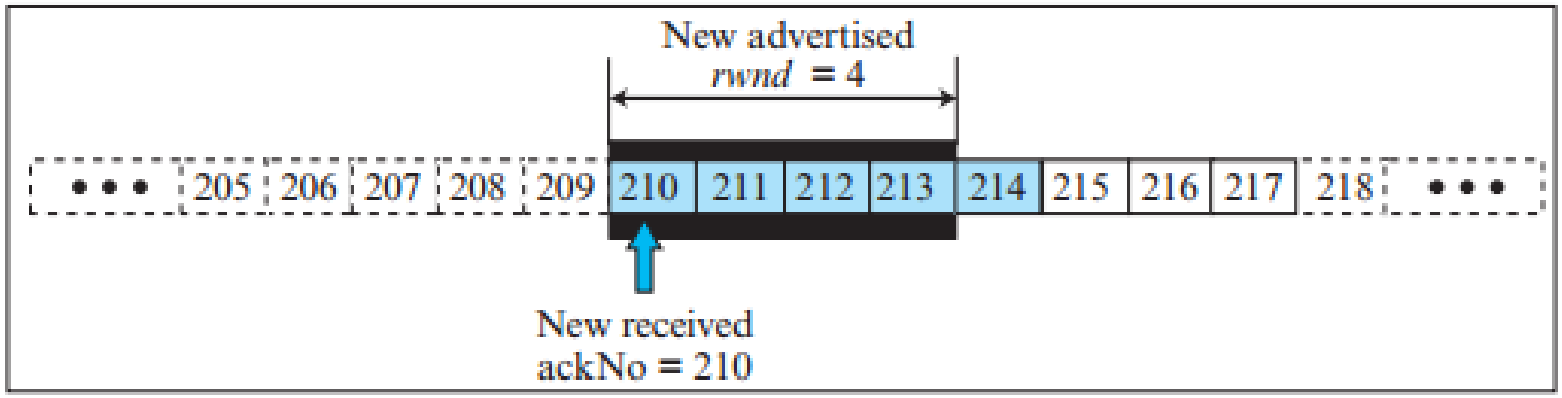
Very small TCP segments waste network bandwidth.

- Caused by slow sender.
- Caused by slow receiver.
- Example:
  - 1 byte of user data
  - 40-byte TCP/IP header
- Results in very poor efficiency.

Figure 3.58 *Example 3.18*



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Nagle's Algorithm (Sender Solution)

Nagle's Algorithm

1. Send the first segment immediately.
2. Buffer subsequent data.
3. Transmit when:
   - ACK is received, or
   - Enough data fills one maximum-size segment.

Advantage

Reduces the number of very small TCP segments while adapting to both application speed and network speed.

Receiver Solution: Clark's Solution

Receiver-Side Solution

- Advertise **rwnd = 0** until
  - Enough space for one maximum-size segment, or
  - At least half of the receive buffer becomes free.
- Another approach is **Delayed ACK**.
- ACK delay should not exceed **500 ms**.

Summary

- Sender solution → Nagle's Algorithm.
- Receiver solution → Clark's Solution.
- Delayed ACK reduces traffic while avoiding silly window syndrome.

TCP Error Control

Goal

TCP guarantees reliable, ordered, and duplicate-free delivery of data.

- Detects corrupted segments.
- Retransmits lost segments.
- Stores out-of-order segments.
- Discards duplicate segments.

TCP Error Control Tools

Checksum | Acknowledgment | Timeout

Checksum and Acknowledgments

Checksum

- 16-bit mandatory field.
- Detects corrupted TCP segments.
- Corrupted segments are discarded.

Acknowledgments

- Confirm successful reception.
- ACK segments do not consume sequence numbers.
- ACK segments are never acknowledged.

Types of Acknowledgments

Cumulative ACK

- Original TCP mechanism.
- Acknowledges all bytes up to the next expected byte.
- Uses ACK field in TCP header.

Selective ACK (SACK)

- Reports out-of-order blocks.
- Reports duplicate blocks.
- Implemented as a TCP option.

Key Point

TCP mainly uses cumulative ACKs; modern TCP also supports SACK.

Generating Acknowledgments

1. Piggyback ACK whenever possible.
2. Delay ACK (up to 500 ms) if only one in-order segment is waiting.
3. Send ACK immediately after two in-order segments.
4. Send duplicate ACK for out-of-order segments.
5. ACK immediately when the missing segment arrives.
6. Discard duplicate segments but send ACK again.

Retransmission

1. **Retransmission after Timeout (RTO)**
   - Sender maintains one retransmission timer.
   - If timer expires, retransmit the oldest unacknowledged segment.
2. **Fast Retransmission**
   - Triggered after three duplicate ACKs.
   - No need to wait for timeout.
   - Improves TCP performance.

Out-of-Order Segments and TCP FSM

Out-of-Order Segments

- Stored temporarily.
- Never delivered to the application until missing data arrive.

TCP FSM

- Sender FSM
- Receiver FSM
- Similar to Selective Repeat (SR).
- With cumulative ACKs, TCP also resembles Go-Back-N (GBN).

Figure 3.59 *Simplified FSM for the TCP sender side*

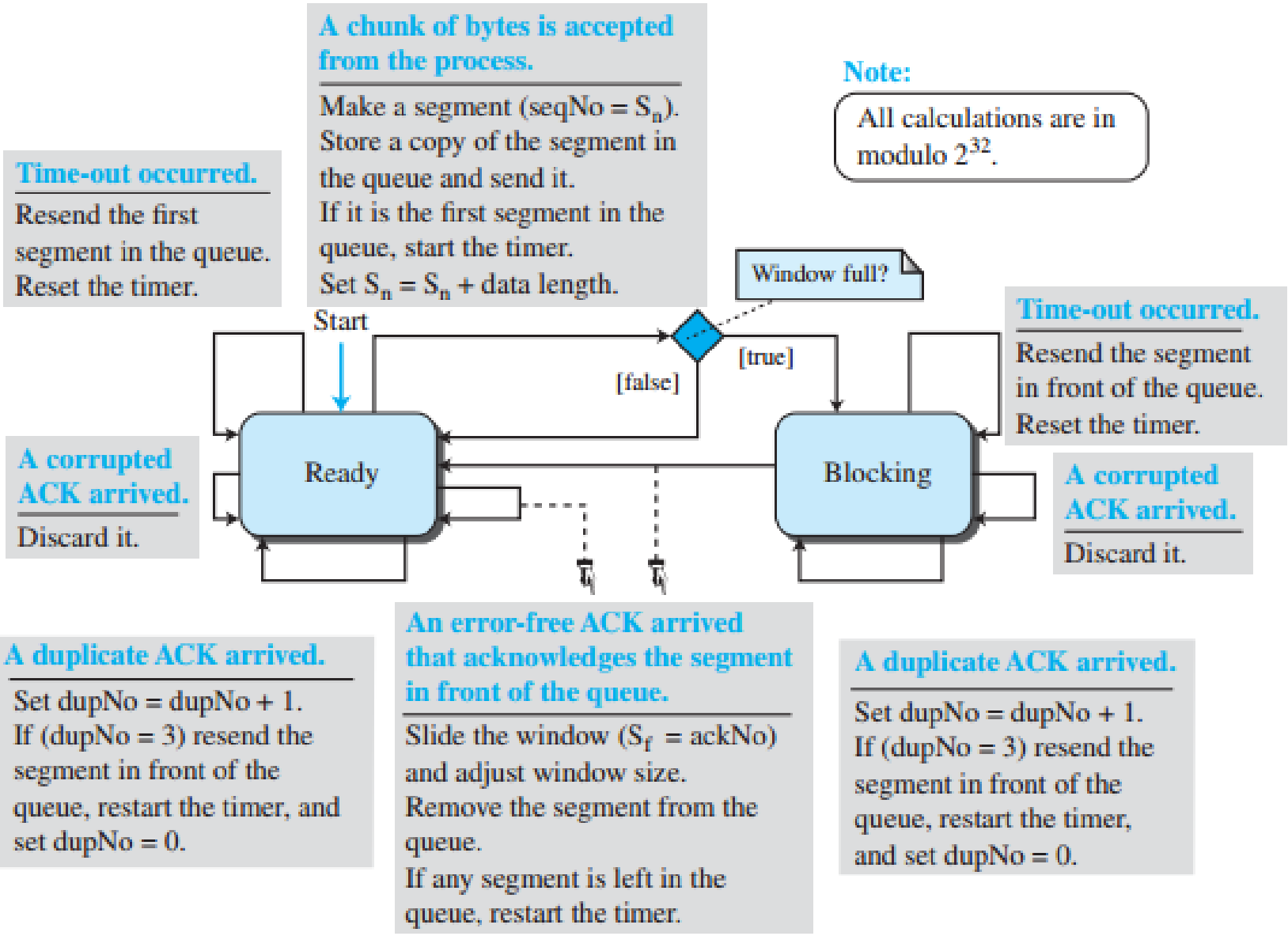
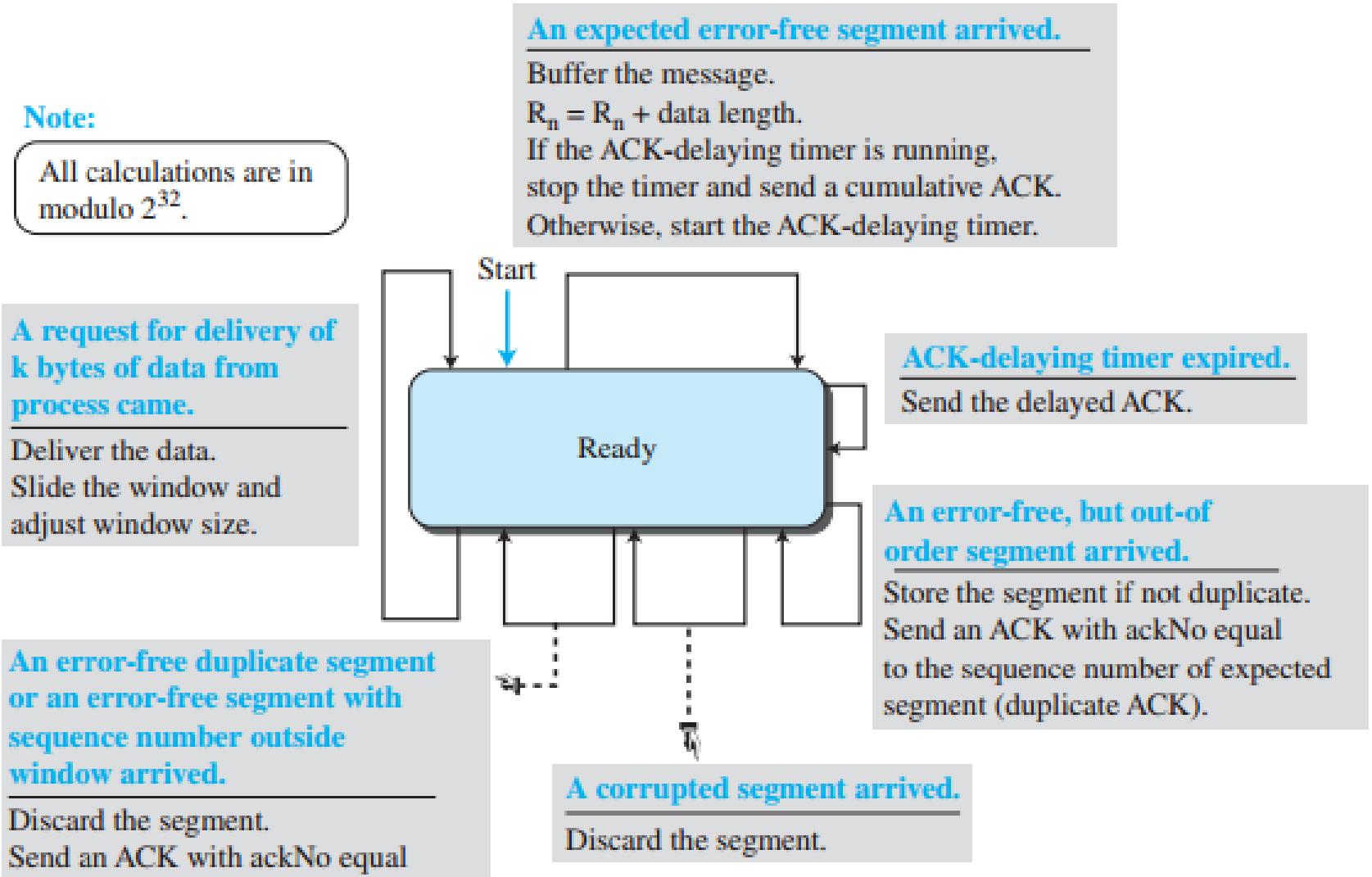


Figure 3.60 *Simplified FSM for the TCP receiver side*



Error Control Scenarios

- Normal operation
- Lost segment
- Fast retransmission
- Delayed segment
- Duplicate segment

Figure 3.61 *Normal operation*

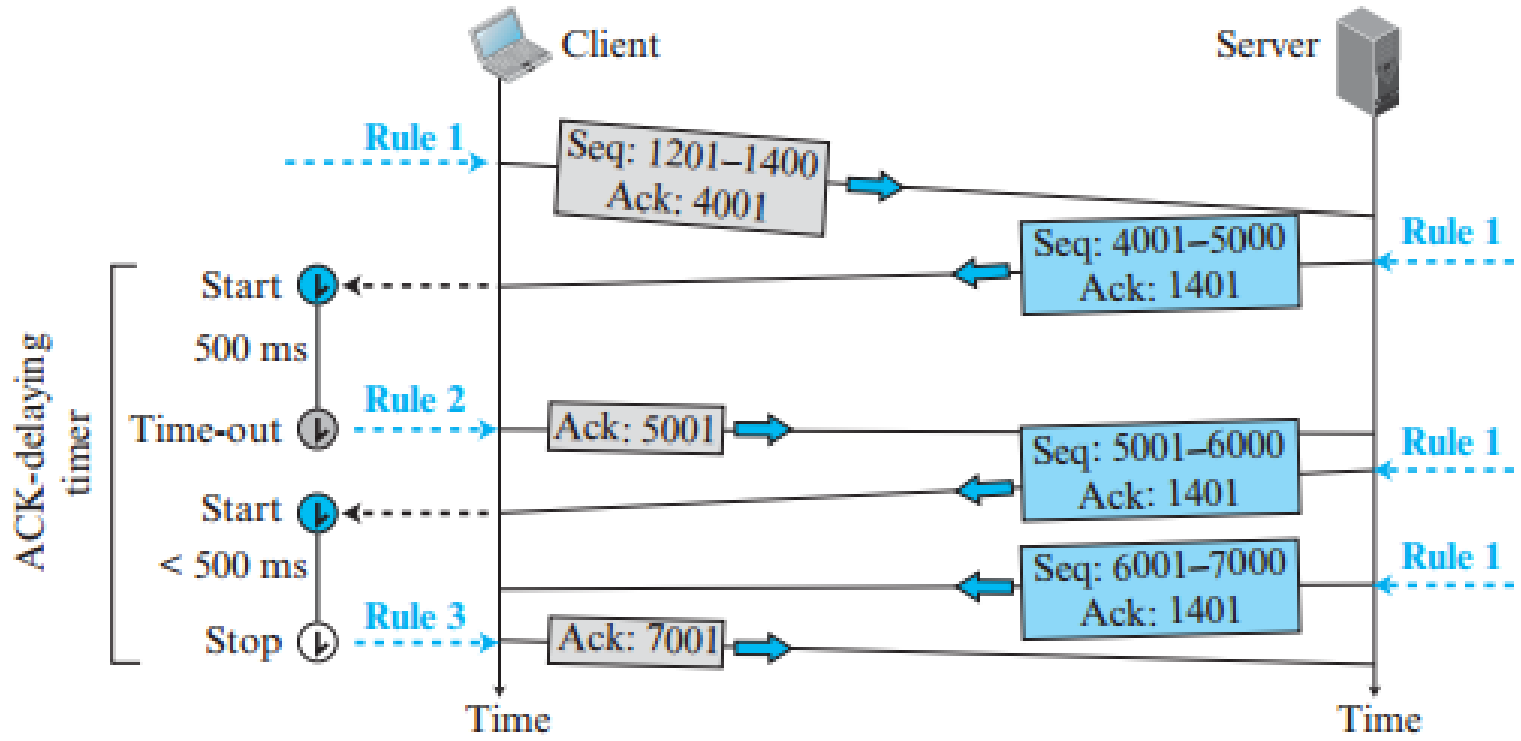


Figure 3.62 *Lost segment*

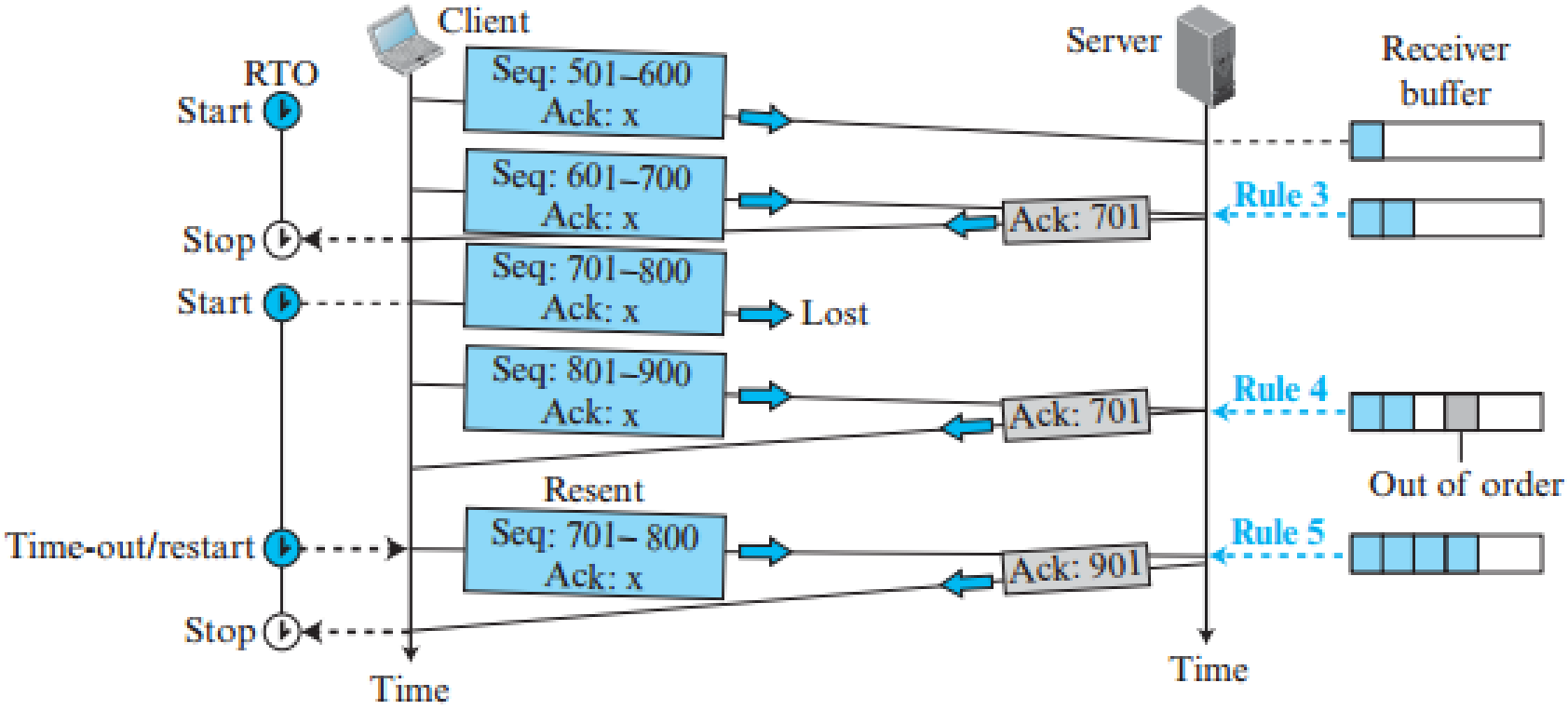
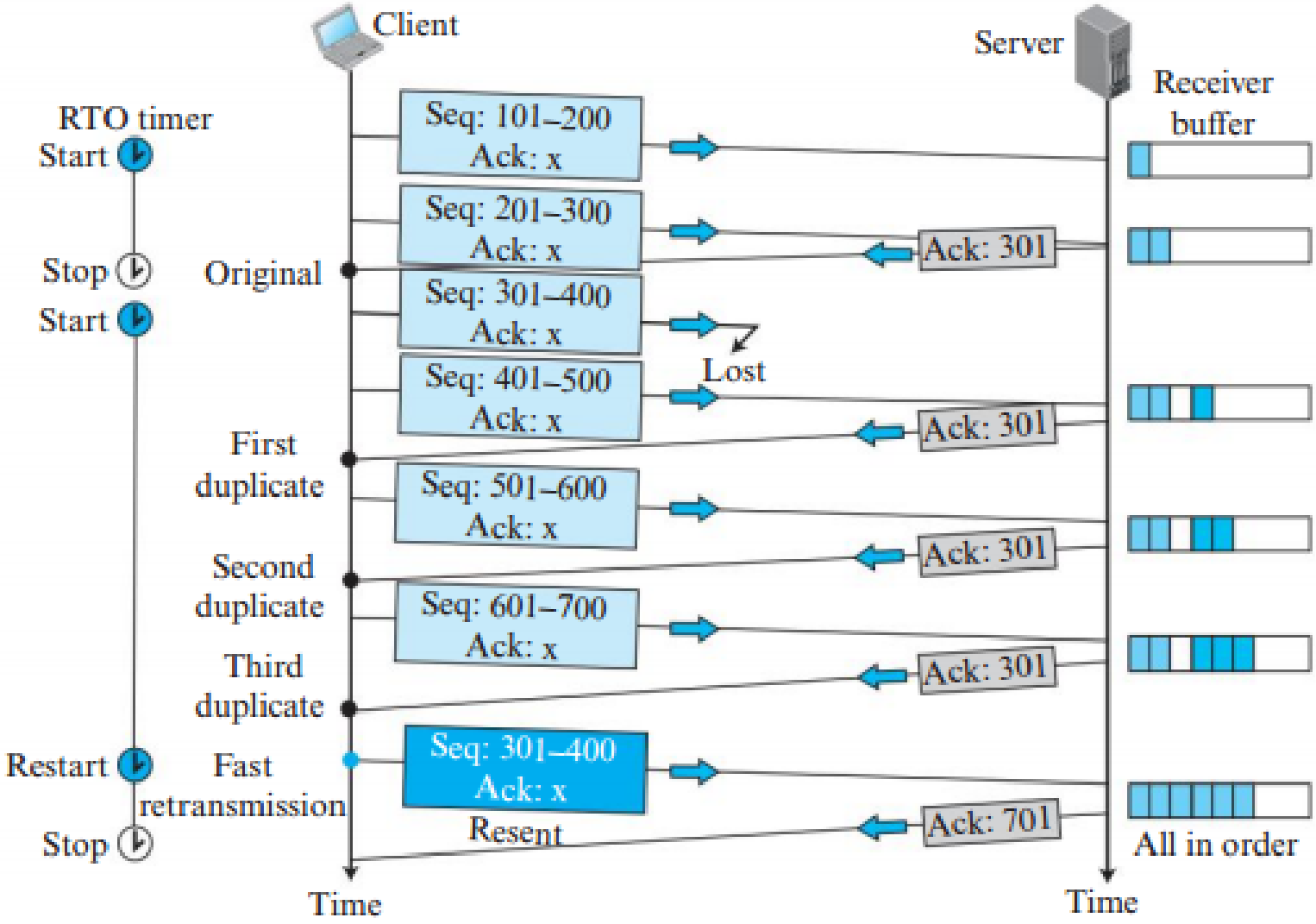


Figure 3.63 *Fast retransmission*



Lost Acknowledgments

Automatic Recovery

- Cumulative ACK automatically corrects many lost ACKs.
- Sender eventually receives a later ACK covering all previous data.

If Recovery Fails

- Retransmission timer expires.
- Sender retransmits the segment.
- Receiver discards duplicate and resends ACK.

Figure 3.64 *Lost acknowledgment*

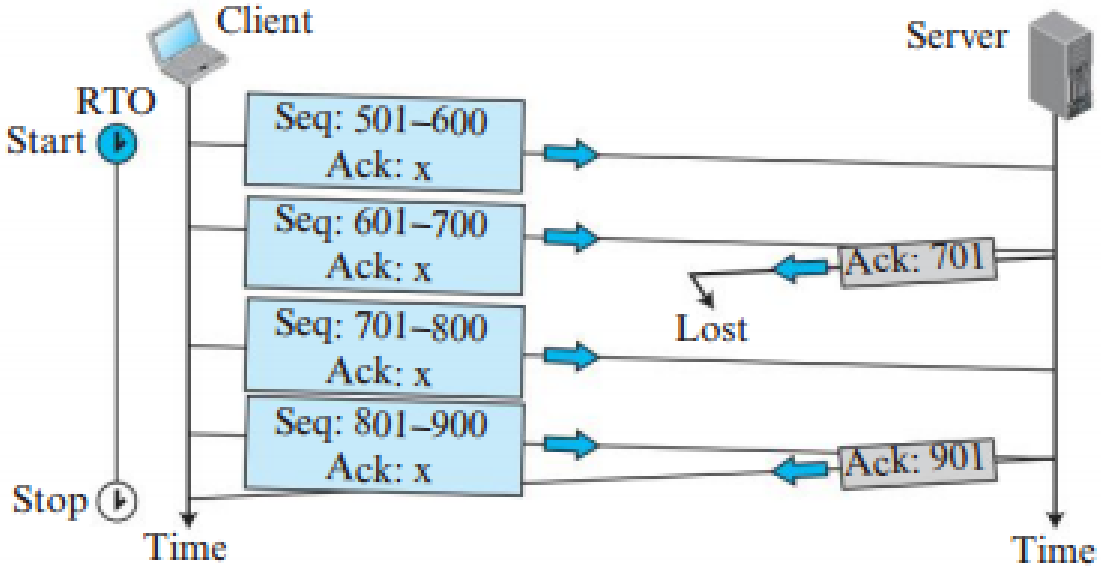
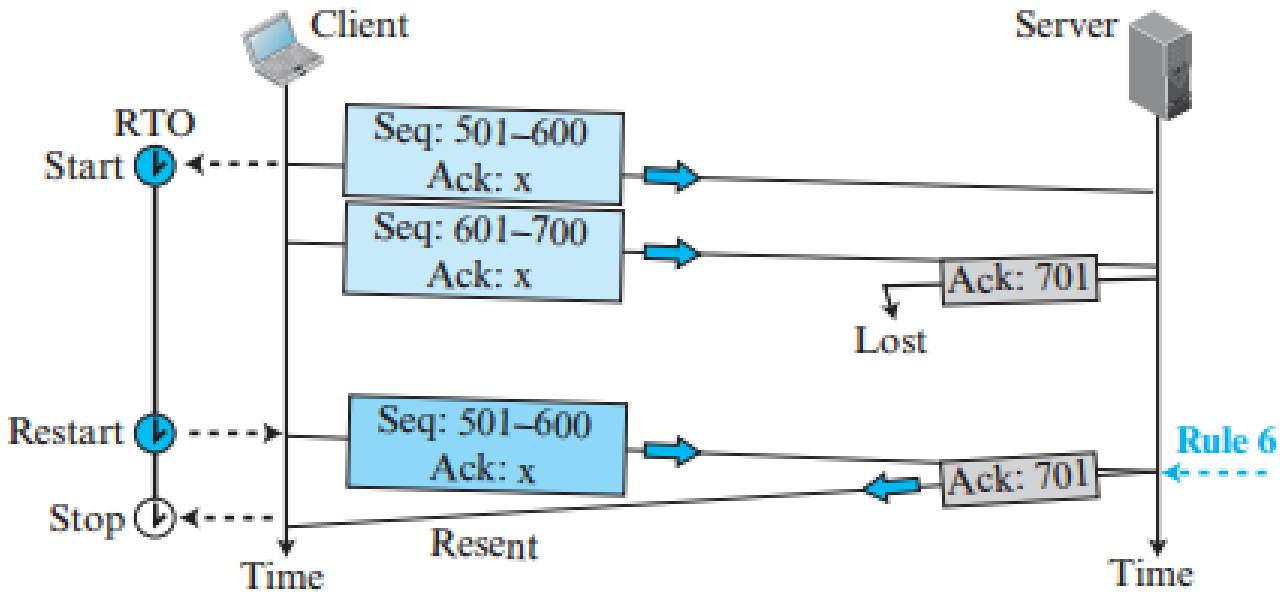


Figure 3.65 *Lost acknowledgment corrected by resending a segment*



Deadlock and Summary

Deadlock

- Receiver advertises **rwnd = 0**.
- Later advertises a non-zero window.
- If this ACK is lost, both sender and receiver wait forever.
- TCP uses a **Persistence Timer** to avoid deadlock.

Summary

- Checksum detects errors.
- ACK confirms delivery.
- Timeout and Fast Retransmission recover losses.
- TCP ensures reliable, ordered delivery.

TCP Congestion Control

Goal

Prevent congestion inside the network while utilizing the available bandwidth efficiently.

- Flow control protects the receiver.
- Congestion control protects the network.
- TCP adjusts its transmission rate according to network congestion.
- Uses two windows:
  - Receive Window (**rwnd**)
  - Congestion Window (**cwnd**)

$$\boxed{\text{Send Window} = \min(\textbf{rwnd}, \textbf{cwnd})}$$

Congestion Detection

TCP detects congestion using:

1. **Timeout (RTO)**
   - Strong indication of congestion.
   - Missing ACK before timeout.
2. **Three Duplicate ACKs**
   - Indicates one segment is missing.
   - Usually a mild congestion.

Key Point

TCP detects congestion only by observing ACKs.

Slow Start

- Initial congestion window:

$$cwnd = 1\ MSS$$

- Each ACK increases:

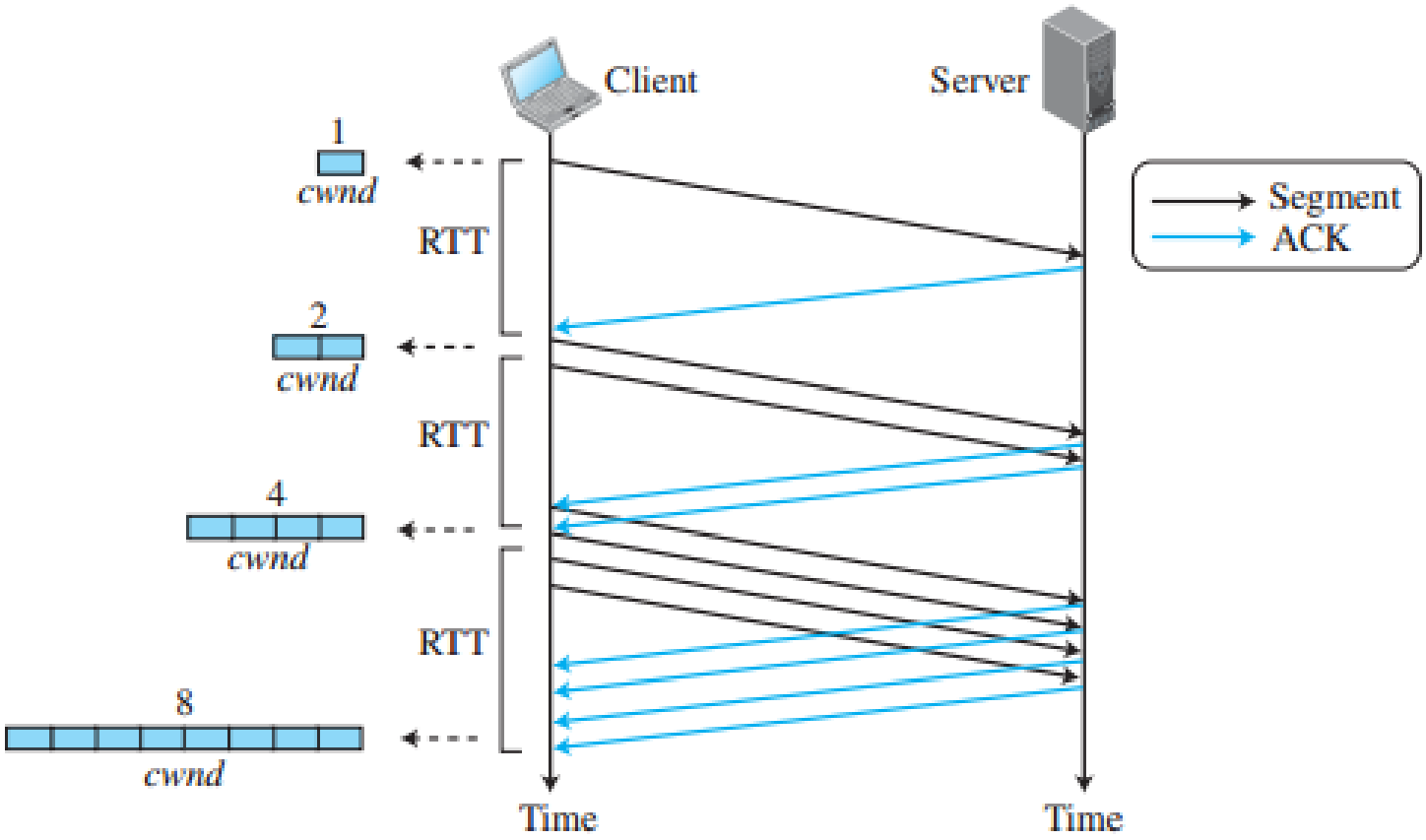
$$cwnd = cwnd + 1$$

- Growth is exponential (per RTT).
- Continues until

$$cwnd \geq ssthresh$$

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$$

Figure 3.66 *Slow start, exponential increase*



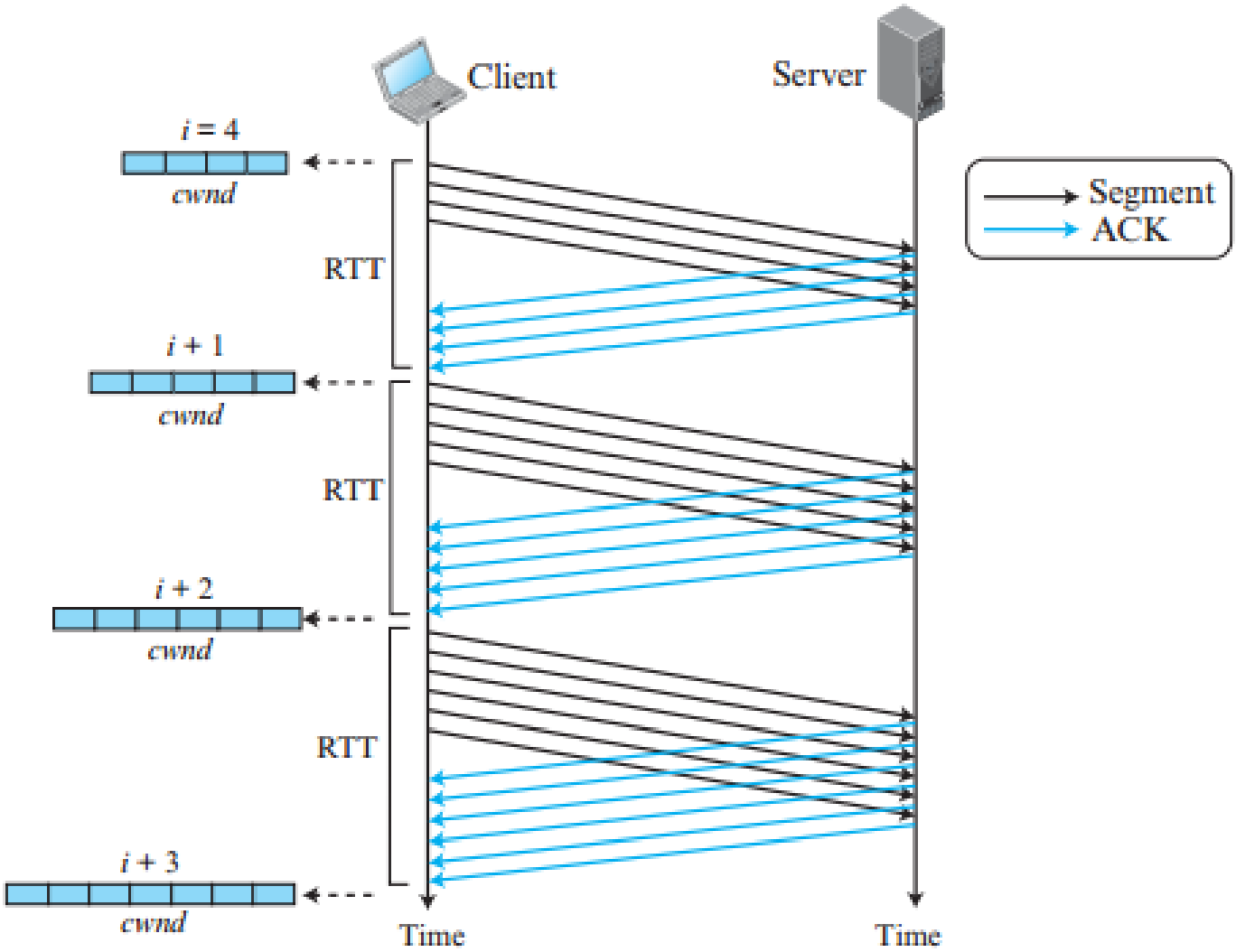
Congestion Avoidance

- Begins after reaching **ssthresh**.
- Growth becomes additive instead of exponential.
- After one RTT,

$$cwnd = cwnd + 1$$

- Prevents congestion before it occurs.

Figure 3.67 *Congestion avoidance, additive increase*



Congestion Control Policies

Policy	**Growth**	**Purpose**
Slow Start	Exponential	Quickly utilize bandwidth
Congestion Avoidance	Additive	Avoid future congestion
Fast Recovery	Fast recovery after 3 duplicate ACKs	Avoid restarting from scratch

TCP Versions

Version	**Characteristics**
Tahoe	Slow Start + Congestion Avoidance. Timeout and 3 duplicate ACKs treated the same.
Reno	Introduced Fast Recovery. Treats timeout and duplicate ACKs differently.
NewReno	Improves Reno by recovering multiple packet losses more efficiently.

Figure 3.68 *FSM for Taho TCP*

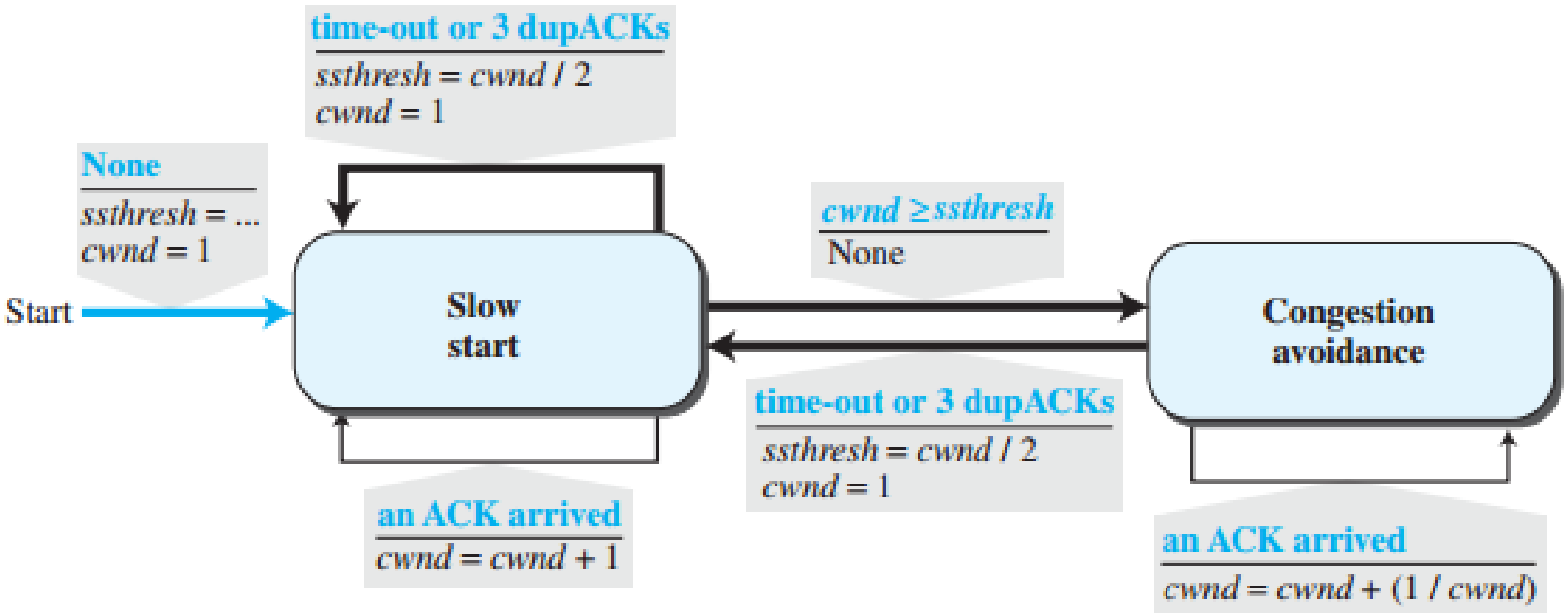


Figure 3.69 *Example of Taho TCP*

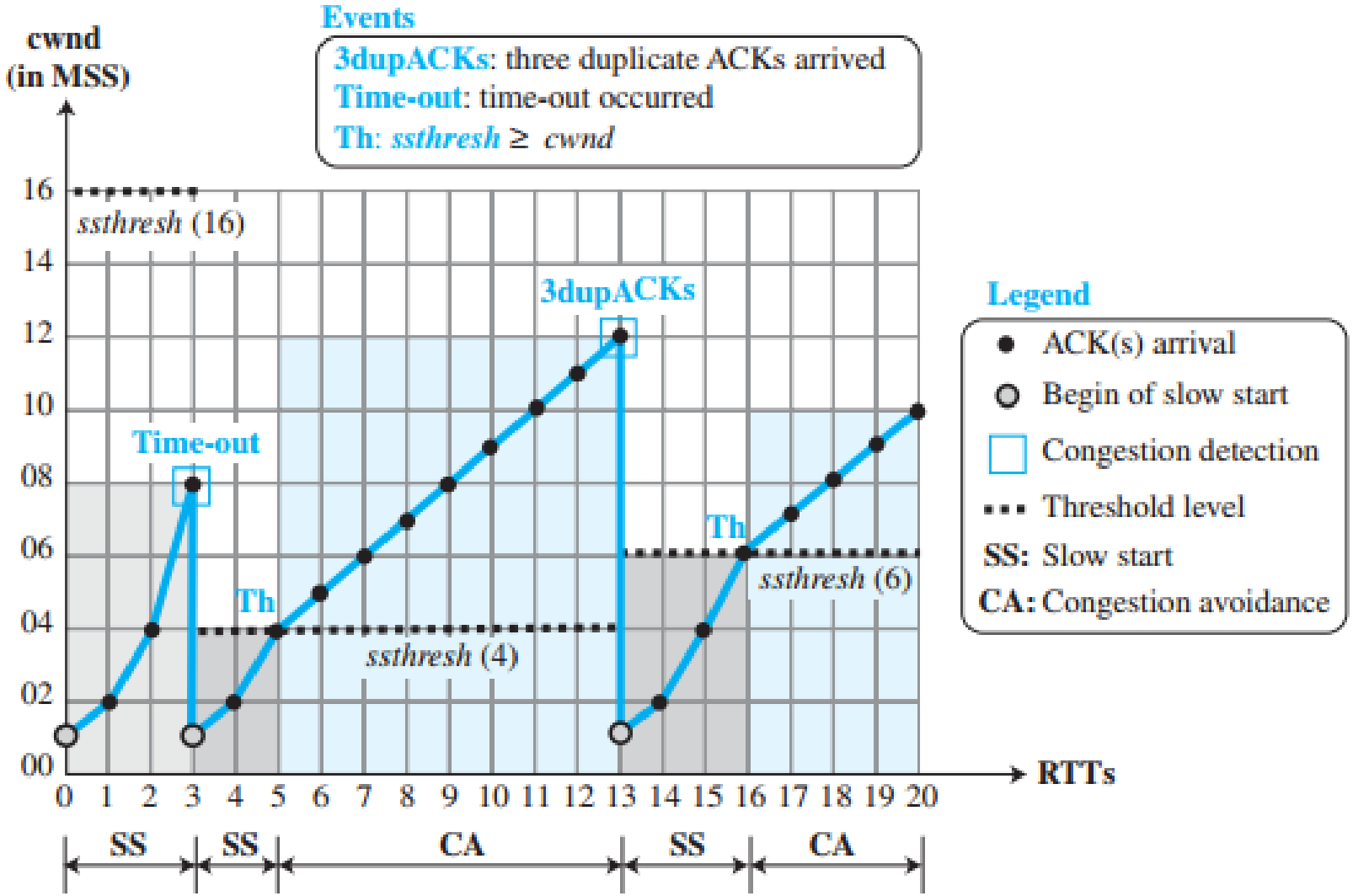


Figure 3.70 *FSM for Reno TCP*

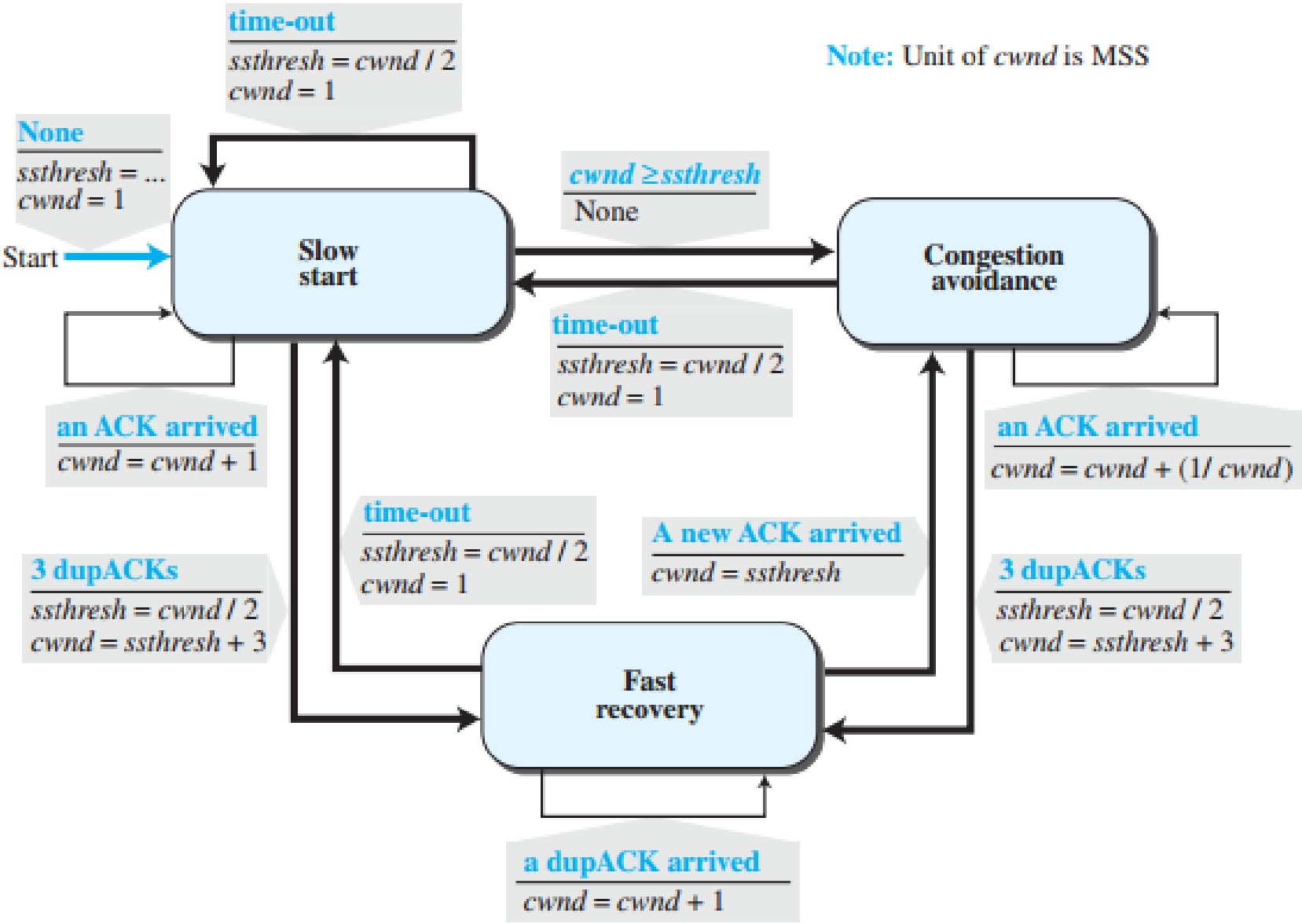
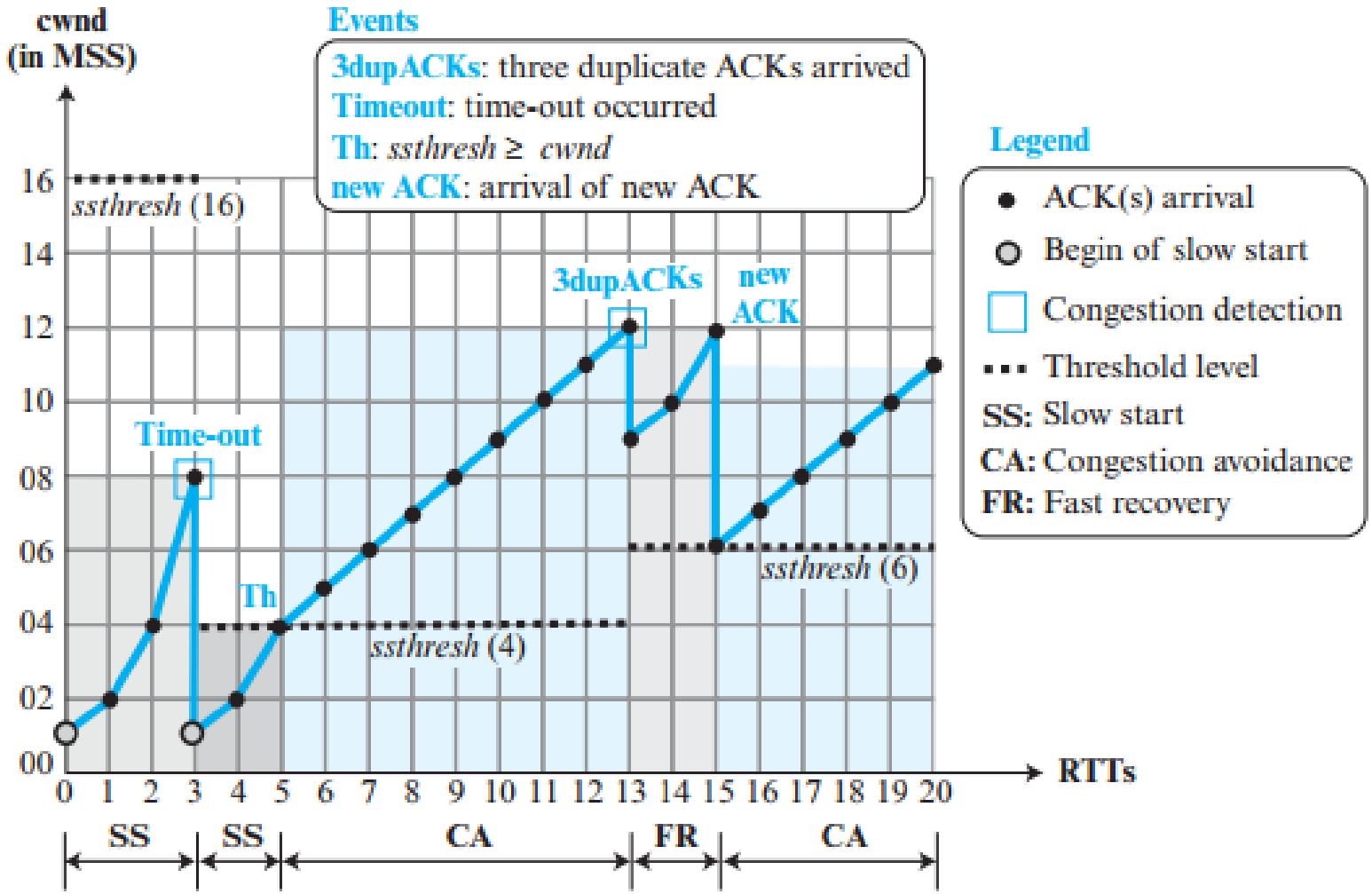


Figure 3.71 *Example of a Reno TCP*



AIMD (Additive Increase Multiplicative Decrease)

- During normal operation:

$$cwnd = cwnd + \frac{1}{cwnd}$$

(Additive Increase)

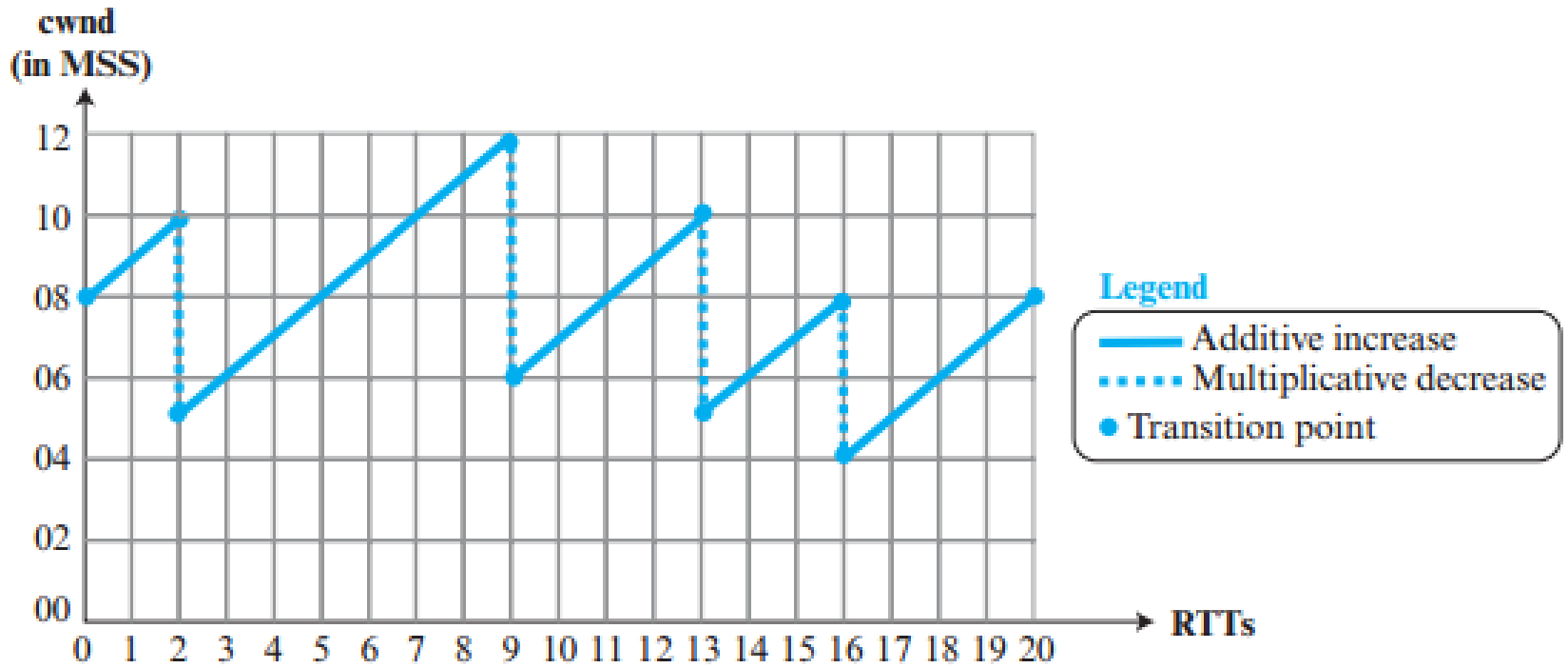
- When congestion occurs:

$$cwnd = \frac{cwnd}{2}$$

(Multiplicative Decrease)

- Produces the characteristic **saw-tooth pattern**.

Figure 3.72 *Additive increase, multiplicative decrease (AIMD)*



TCP Throughput

Approximate Throughput

$$\text{Throughput} = \frac{0.75\, W_{max}}{RTT}$$

Where,

- W_{max} = Average congestion window before congestion.
- RTT = Round Trip Time.
- MSS = Maximum Segment Size.

Key Points

- Larger congestion window ⇒ Higher throughput.
- Larger RTT ⇒ Lower throughput.
- TCP continuously adapts its transmission rate.

TCP Timers

Purpose

TCP uses timers to ensure reliable communication, recover from errors, and prevent deadlocks.

Major TCP Timers

1. Retransmission Timer
2. Persistence Timer
3. Keepalive Timer
4. TIME-WAIT Timer

Retransmission Timer

Purpose

Retransmits lost TCP segments.

Operation

1. Start timer when the first unacknowledged segment is sent.
2. If timer expires, retransmit the oldest unacknowledged segment.
3. Remove acknowledged segments from the queue.
4. Stop the timer when the queue becomes empty.

Key Point

TCP maintains one retransmission timer for each connection.

RTT and Retransmission Timeout (RTO)

Measured RTT

Time between sending a segment and receiving its ACK.

Smoothed RTT

$$RTTS = (1 - \alpha)RTTS + \alpha RTTM$$

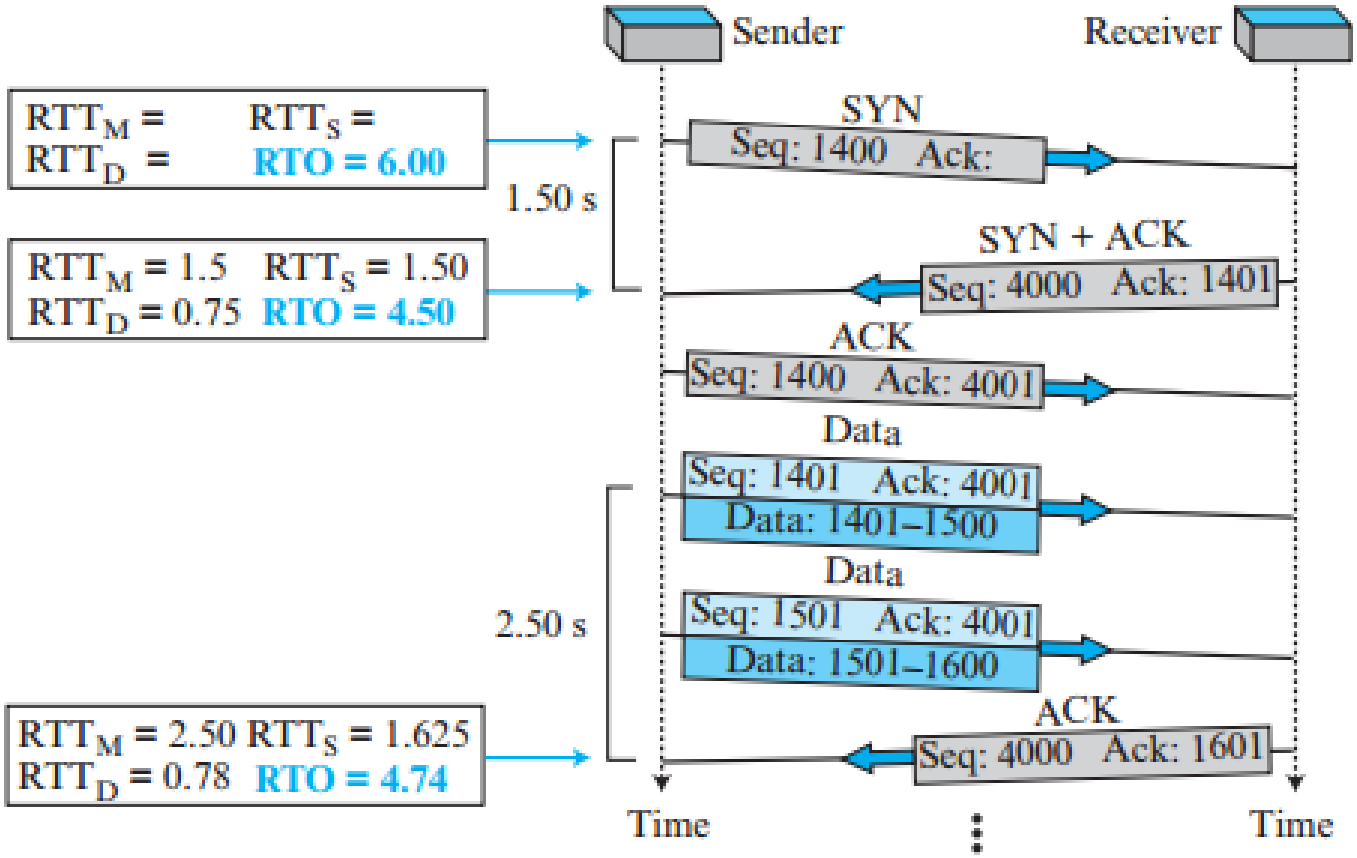
RTT Deviation

$$RTTD = (1 - \beta)RTTD + \beta|RTTS - RTTM|$$

Retransmission Timeout

$$\boxed{RTO = RTTS + 4 \times RTTD}$$

Figure 3.73 *Example 3.22*



Karn's Algorithm and Exponential Backoff

Karn's Algorithm

- Ignore RTT measurements for retransmitted segments.
- Update RTT only after successful transmission without retransmission.

Exponential Backoff

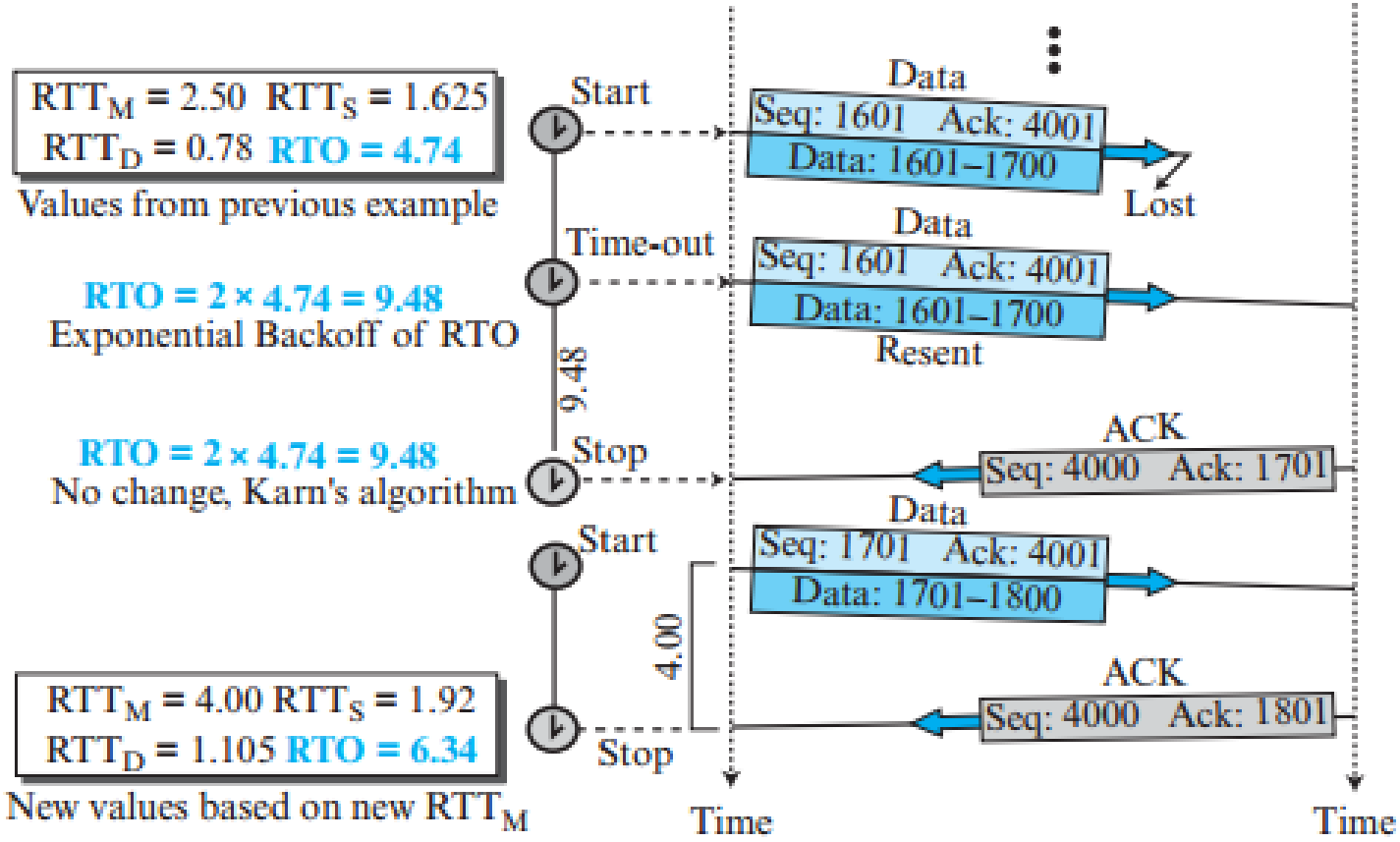
$$RTO \leftarrow 2 \times RTO$$

after every retransmission.

Purpose

Avoid unnecessary retransmissions during heavy congestion.

Figure 3.74 *Example 3.23*



Persistence Timer

Problem

Receiver advertises

$$rwnd = 0$$

and later advertises a non-zero window, but the ACK is lost.

Solution

- Start Persistence Timer.
- Send a 1-byte probe segment when timer expires.
- Receiver replies with updated window size.
- Probe interval doubles until a maximum of about 60 seconds.

Key Point

Prevents deadlock caused by a zero-window advertisement.

Keepalive and TIME-WAIT Timers

Keepalive Timer

- Detects idle or crashed peers.
- Sends keepalive probes.
- Closes inactive connections when no response is received.

TIME-WAIT Timer

- Starts after connection termination.
- Waits for 2MSL.
- Ensures delayed segments disappear.
- Allows retransmission of the final ACK if necessary.

Summary

- Retransmission Timer $\rightarrow$ Recover lost segments.
- Persistence Timer $\rightarrow$ Prevent deadlock.
- Keepalive Timer $\rightarrow$ Detect idle connections.
- TIME-WAIT Timer $\rightarrow$ Ensure reliable connection termination.